

CS5284 : Graph Machine Learning

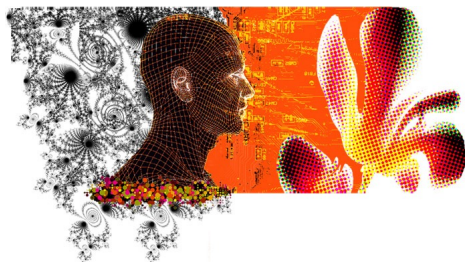
Lecture 10 : Graph Transformers

Semester 1 2025/26

Xavier Bresson

<https://x.com/xbresson>

Department of Computer Science
National University of Singapore (NUS)



Course lectures

- Introduction to Graph Machine Learning
- Part 1: GML without feature learning (before 2014)
 - Introduction to Graph Science
 - Graph Analysis Techniques without Feature Learning
 - Graph clustering
 - Graph SVM
 - Recommendation on graphs
 - Graph-based visualization
- Part 2 : GML with shallow feature learning (2014-2016)
 - Shallow graph feature learning
- Part 3 : GML with deep feature learning, a.k.a. GNNs (after 2016)
 - Graph Convolutional Networks (spectral and spatial)
 - Weisfeiler-Lehman GNNs
 - • Graph Transformer
 - Graph generation & molecular science
 - Material design
 - Integrating GNNs and LLMs
 - Deep Learning for Combinatorial optimization

Outline

- Review of graph network architectures
- Graph positional encoding  Gives a canonical ordering to each node
- Transformers for sequences
- Graph Transformers
- ViT/MLP-Mixer for images
- Graph ViT/MLP-Mixer
- Conclusion

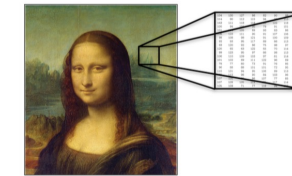
Outline

- Review of graph network architectures
- Graph positional encoding
- Transformers for sequences
- Graph Transformers
- ViT/MLP-Mixer for images
- Graph ViT/MLP-Mixer
- Conclusion

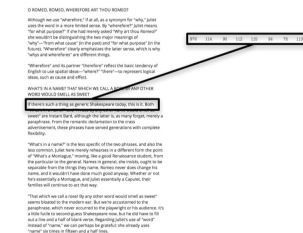
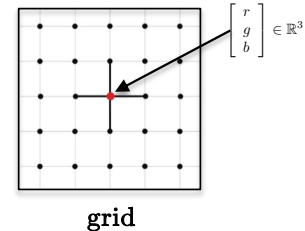
Neural network architectures

- To enable DL+X, we need three key ingredients :
 - Data, GPU and neural network
- There are four classes of networks
 - CNNs/RNNs/Transformers are designed for grids, sequences and sets.
 - GPUs/DL libraries PyTorch^[1], TensorFlow^[2] are optimized for these architectures.
 - Graph Neural Networks (GNNs) are more universal and broadly applicable architectures.
 - GPUs are not optimized for sparse linear algebra.

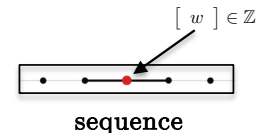
↓
aka for graphs



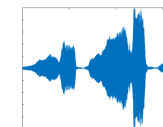
Computer Vision (CV)



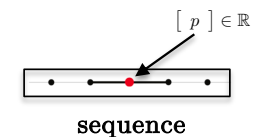
Natural Language Processing (NLP)



Graph Analysis



Speech Recognition (SR)



[1] Paszke et-al, Automatic differentiation in pytorch, 2017

[2] Abadi et-al, TensorFlow: a system for Large-Scale machine learning, 2016

GNN architectures

→ wrt node ordering, ie regardless how we index each node/edge (e.g. 18/69/999), the learned representation of the nodes/edges/graph is the same

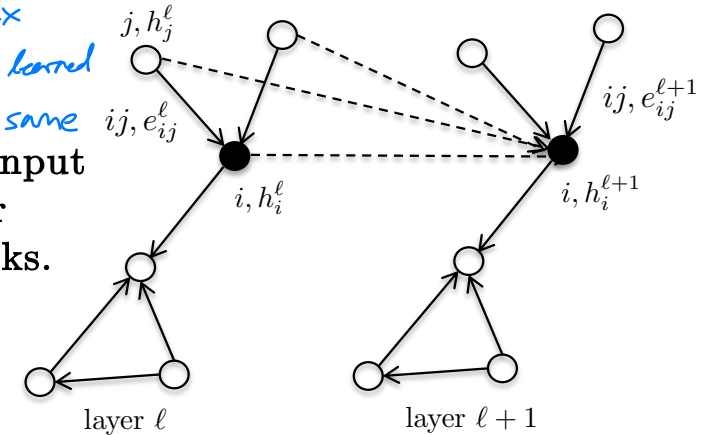
What is a graph network?

- Compositional parametric equivariant function that maps an input pair (graph, feature) to continuous vectorial representation for node/edge/graph which can be used for downstream graph tasks.
- Often defined as a message-passing process^[1]:

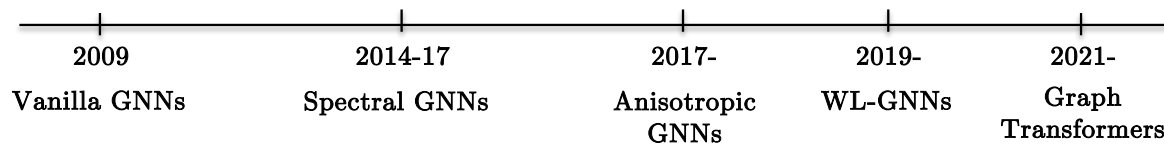
$$\text{Node-update: } h_i^{\ell+1} = f_{\text{node}}(h_i^{\ell}, \{h_j^{\ell}, e_{ij}^{\ell} : j \in \mathcal{N}_i\}) \in \mathbb{R}^d$$

$$\text{Edge-update: } e_{ij}^{\ell+1} = f_{\text{edge}}(e_{ij}^{\ell}, h_i^{\ell}, h_j^{\ell}) \in \mathbb{R}^d$$

↑
edge_{ij} ↑ node_i ↑ node_j



Brief history of graph networks :



Powerful GNNs
Expressivity, generalization,
Linear complexity

Transferable

exception:
Transformers

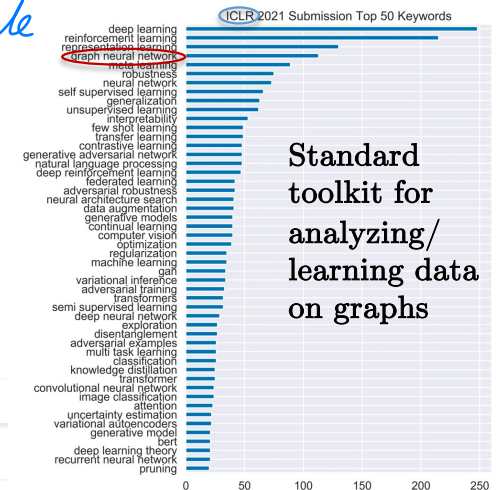
[1] Gilmer, Schoenholz, Riley, Vinyals, Dahl, Neural message passing for quantum chemistry, 2017

Google

"graph neural networks"

All Images Videos News

About 1,260,000 results (0.61 seconds)



Vanilla GNNs

Contributions

- First NN^[1] with layers **equivariant/invariant w.r.t. index permutation, independent to neighborhood and graph size, local reception field, weight sharing.**
- Linear complexity $O(N+E)$

Limitations

- Simple model (vanilla RNN w/ aggregation of neighbors)
- Vanishing gradient problem

Didn't work in practice as DL needs many tricks:

- Layer normalization**: otherwise the values in deep layers will vanish/explode
- For deep architecture, **residual connections** are a must

$$h_i^{\ell+1} = f_{\text{VGNN}}(h_i^{\ell}, \{h_j^{\ell} : j \in \mathcal{N}_i\})$$

$$= \sum_{j \rightarrow i} \sigma(Ux_i + Vh_j^{\ell}) \in \mathbb{R}^d$$

[1] Scarselli, Gori, Tsoi, Hagenbuchner, Monfardini, The Graph Neural Network Model, 2009

The Graph Neural Network Model

Franco Scarselli, Marco Gori, *Fellow, IEEE*, Ah Chung Tsoi, Markus Hagenbuchner, *Member, IEEE*, and Gabriele Monfardini

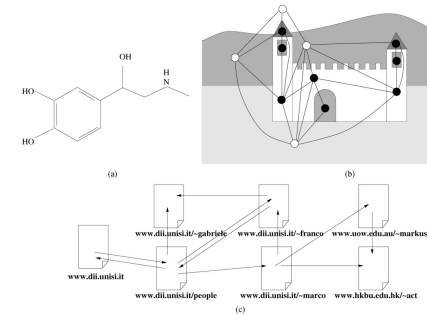


Fig. 1. Some applications where the information is represented by graphs: (a) a chemical compound (adrenaline), (b) an image, and (c) a subset of the web.

Spectral GNNs

Contributions^[1,2,3]

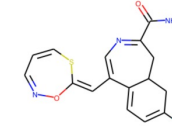
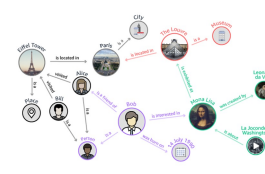
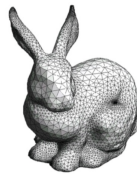
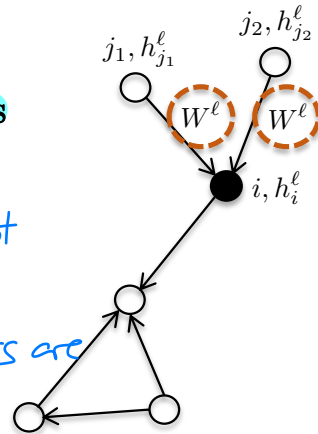
- Leverage graph spectral theory to define **convolution on graphs**
- Stable, robust to perturbation, **exact k-hop kernel support**
- Linear complexity **O(N+E)**
- GCN^[3] is the most popular GNN technique

Limitation

- **Isotropic kernels**

↓
expressivity is not good enough
most real world datasets are anisotropic, except:

- ① Cosmology
- ② Existence of tumors (Orientation of tumor does not matter)



Most data is anisotropic

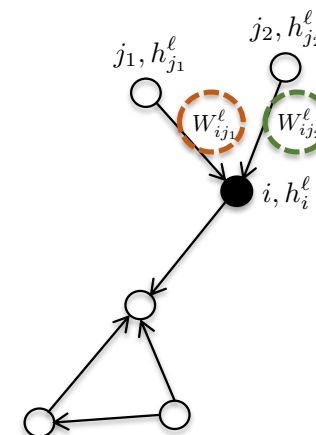
$$\begin{aligned}
 h^{\ell+1} &= \sigma(s^{\ell} *_G h^{\ell}) \in \mathbb{R}^{N \times d} \\
 &= \sigma(U \hat{s}^{\ell}(\Lambda) U^T h^{\ell}) \\
 &= \sigma(\hat{s}^{\ell}(\Delta) h^{\ell}), \quad \Delta = U \Lambda U^T \\
 &= \sigma(\sum_{k=0}^{K-1} T_k^{\text{Chebyshev}}(\Delta) h^{\ell} W_k^{\ell}) \\
 &= \sigma(D^{-1/2} A D^{-1/2} h^{\ell} W^{\ell}) \\
 \text{1st order approx} \\
 h_i^{\ell+1} &= \sigma(\frac{1}{\sqrt{d_i}} \sum_{j \in \mathcal{N}_i} \frac{1}{\sqrt{d_j}} A_{ij} W^{\ell} h_j^{\ell}) \in \mathbb{R}^d
 \end{aligned}$$

- [1] Bruna, Zaremba, Szlam, LeCun, Spectral networks and locally connected networks on graphs, 2013 (6,400 citations)
- [2] Defferrard, Bresson, Vandergheynst, Convolutional neural networks on graphs with fast localized spectral filtering, 2016 (9,800 citations)
- [3] Kipf, Welling, Semi-supervised classification with graph convolutional networks, 2017 (39,000 citations)

Anisotropic GNNs

- Contributions

- Design anisotropic equivariant mechanisms that **treat neighbors differently**.
- GatedGCNs^[1] with PDE-based anisotropic diffusion^[2], GAT^[3] with softmax attention^[4], Mesh CNNs^[5] with anisotropic gauge equivariant kernels
- Stable, robust, **interpretable, linear complexity** $O(N+E)$



Chemistry: molecules
have rings

↑
cannot count the
number of rings
and cliques in graph

↓
Social networks
have many
cliques

$$h_i^{\ell+1} = h_i^{\ell} + \text{ReLU}\left(\text{BN}\left(W_1^{\ell} h_i^{\ell} + \sum_{j \in \mathcal{N}_i} e_{ij}^{\ell} \odot W_2^{\ell} h_j^{\ell}\right)\right)$$

$$e_{ij}^{\ell} = \frac{\sigma(\hat{e}_{ij}^{\ell})}{\sum_{j' \in \mathcal{N}_i} \sigma(\hat{e}_{ij'}^{\ell}) + \varepsilon}$$

$$\hat{e}_{ij}^{\ell} = \hat{e}_{ij}^{\ell-1} + \text{ReLU}\left(\text{BN}\left(V_1^{\ell} h_i^{\ell-1} + V_2^{\ell} h_j^{\ell-1} + V_3^{\ell} \hat{e}_{ij}^{\ell-1}\right)\right)$$

GatedGCNs^[1]

(based on Perona-Malik's anisotropic PDE^[2] generalized on graphs)



- Limitations

- Low expressivity, over-squashing issue

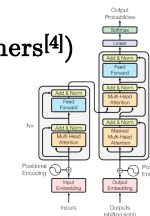
$$h_i^{\ell+1} = \text{Concat}_{k=1}^K \left(\text{ELU} \left(\sum_{j \in \mathcal{N}_i} e_{ij}^{k,\ell} W_1^{k,\ell} h_j^{\ell} \right) \right)$$

$$e_{ij}^{k,\ell} = \text{Softmax}_{\mathcal{N}_i}(\hat{e}_{ij}^{k,\ell}) = \frac{\exp(\hat{e}_{ij}^{k,\ell})}{\sum_{j' \in \mathcal{N}_i} \exp(\hat{e}_{ij'}^{k,\ell})}$$

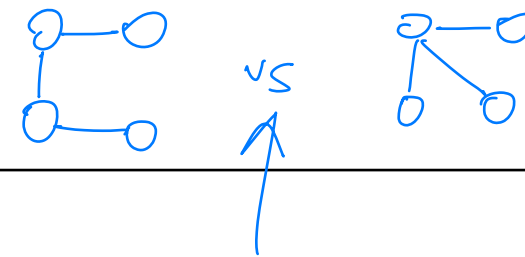
$$\hat{e}_{ij}^{k,\ell} = \text{LeakyReLU}\left(W_2^{k,\ell} \text{Concat}(W_1^{k,\ell} h_i^{\ell}, W_1^{k,\ell} h_j^{\ell})\right)$$

GAT^[3]

(based on Transformers^[4])



- [1] Bresson, Laurent, Residual gated graph convnets, 2017
- [2] Perona, Malik, Scale-space and edge detection using anisotropic diffusion, 1987
- [3] Velickovic, Cucurull, Casanova, Romero, Lio, Bengio, Graph Attention Networks, 2017
- [4] Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin, Attention is all you need, 2017
- [5] De Haan, Weiler, Cohen, Welling, Gauge equivariant mesh CNNs: Anisotropic convolutions on geometric graphs, 2020



Low expressivity

Graphs that have different structure

- Most GNNs s.a. GCN^[1], GAT^[2], GatedGCNs^[3] have theoretically low expressivity/representation power to
 - Distinguish (simple) non-isomorphic graphs^[4,5].
 - Identify/count elementary sub-structures like cycles and cliques^[6].

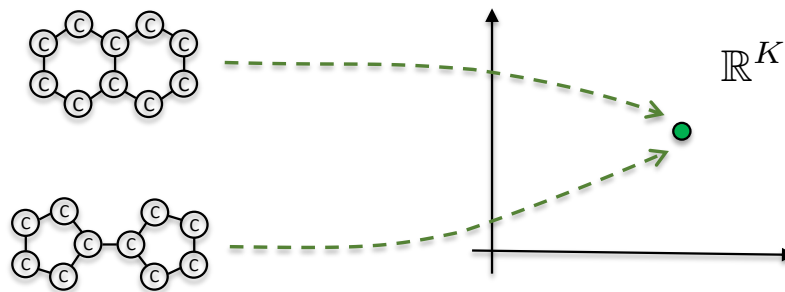


Fig 1. Vectorial graph representation

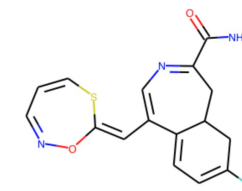


Fig 2. Graph with cycles (rings)

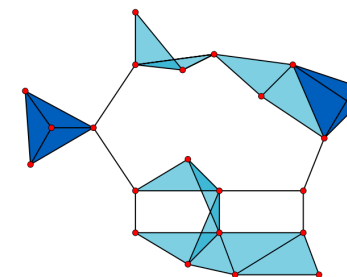


Fig 3. Graph with
19 × 3-vertex cliques (light and dark blue triangles)
and 2 × 4-vertex cliques (dark blue areas)

- [1] Kipf, Welling, Semi-supervised classification with graph convolutional networks, 2017
- [2] Velickovic, Cucurull, Casanova, Romero, Lio, Bengio, Graph Attention Networks, 2018
- [3] Bresson, Laurent, Residual gated graph convnets, 2017
- [4] Xu, Hu, Leskovec, Jegelka, How powerful are graph neural networks? 2019
- [5] Morris, Ritzert, Fey, Hamilton, Lenssen, Rattan, Grohe, Weisfeiler and leman go neural: Higher-order graph networks, 2019
- [6] Chen, Chen, Villar, Bruna, Can graph neural networks count substructures? 2020

Graph isomorphism ^{Not important}

- Graph isomorphism : Two graphs are isomorphic if there exists an index permutation that preserves node neighbors.
- Determining whether two graphs are isomorphic is **NP-intermediate**. It is not known if a polynomial time algorithm exists, or the problem is NP-hard.

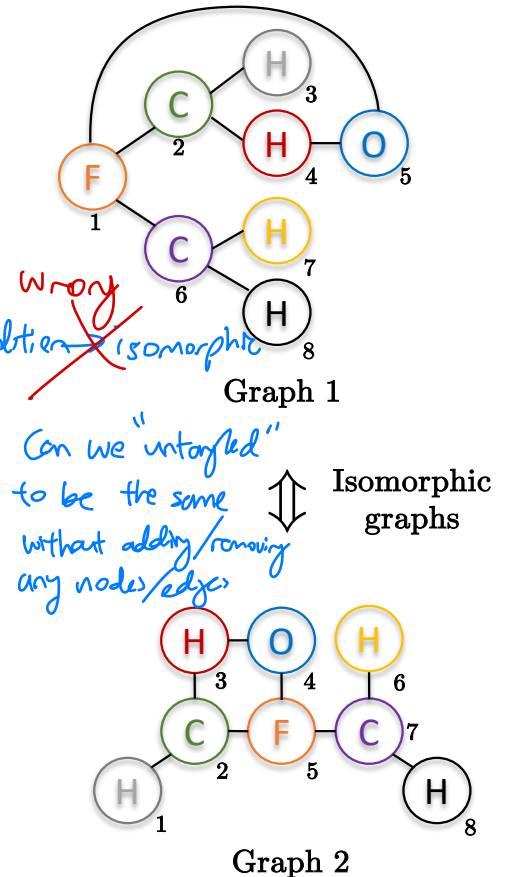
$\neg \text{condition} \rightarrow \neg \text{isomorphic}$ **NOT** condition $\rightarrow \text{isomorphic}$

- Weisfeiler-Lehman test**^[1] provides a **necessary (but not sufficient) condition** to guarantee that two graphs are isomorphic.
 - Design an injective coloring function f_{WL} that takes a pair (node, its neighborhood) as input, and outputs a new node color :

$$f_{WL}(c_i^\ell, \{c_j^\ell\}_{j \in \mathcal{N}_i}) = c_i^{\ell+1}$$

iteration

Multiset (set of unordered and repetitive elements)

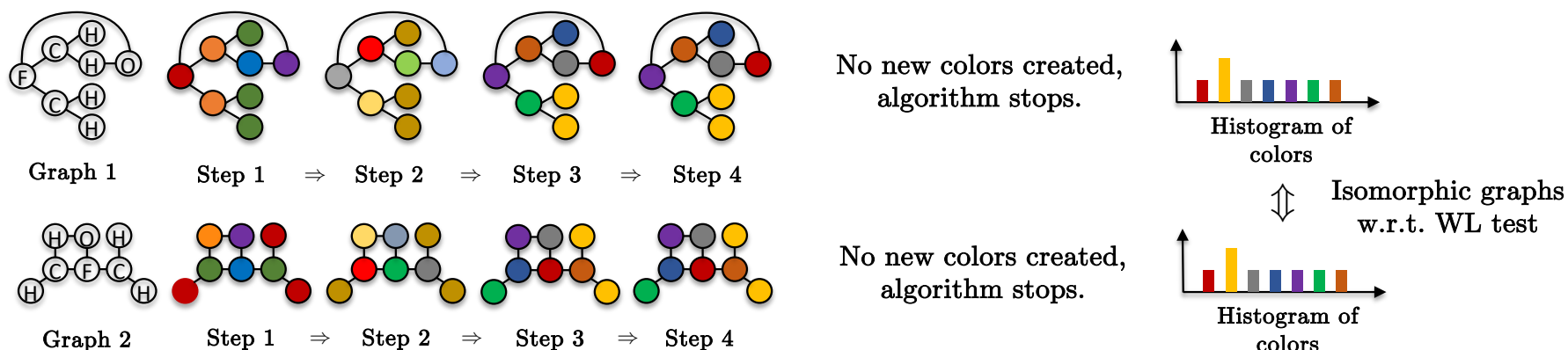


[1] Weisfeiler, Lehman, A reduction of a graph to a canonical form and an algebra arising during this reduction, 1968

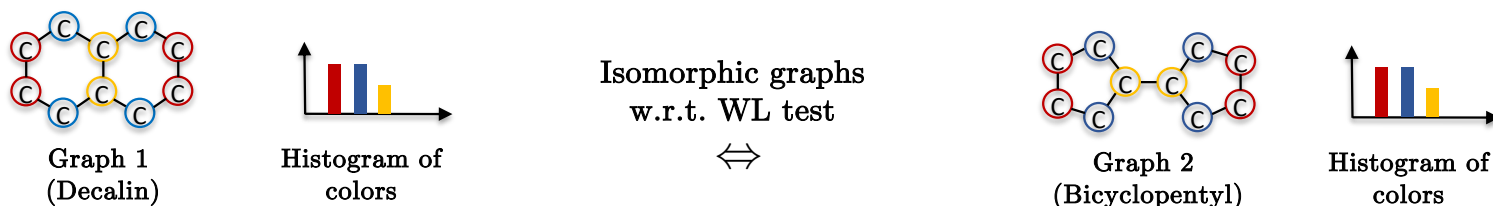
Skipped but examinable

Weisfeiler-Lehman test

- WL algorithm^[1] iteratively applies the coloring function f_{WL} until no new colors are created :



- However the 1-WL test can fail to distinguish (simple) non-isomorphic graphs :

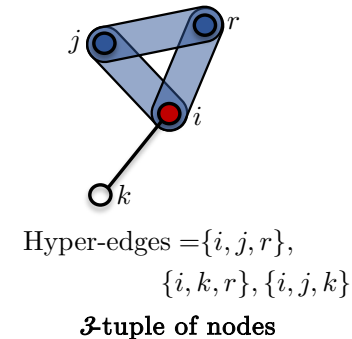
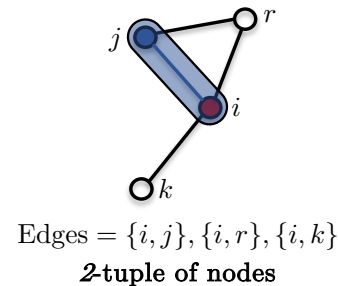
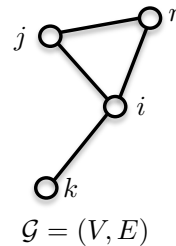


[1] Weisfeiler, Lehman, A reduction of a graph to a canonical form and an algebra arising during this reduction, 1968

skipped

WL-GNNs

- GIN^[1] designed to be as maximally expressive as the original 1-WL test^[2].
- k-WL tests : Original test uses 2-tuple of nodes. To improve expressivity power of WL test, higher-order interactions between nodes with k-tuple of nodes with $k \geq 3$ can be used.
 - k-order equivariant GNNs^[3] are theoretically more expressive but
 - These networks require $O(N^k)$ memory/speed complexities, with at least $k=3$ to be more powerful than GIN, which means $O(N^3)$ and thus not practical.
 - 3-WL/Ring/2-FGNN GNNs^[4,5,6] have $O(N^2)$ -memory but $O(N^3)$ -speed complexities.
 - Expressivity does not necessarily imply generalization^[7].



- [1] Xu, Hu, Leskovec, Jegelka, How powerful are graph neural networks?, 2019
- [2] Weisfeiler, Lehman, A reduction of a graph to a canonical form and an algebra arising during this reduction, 1968
- [3] Maron, Ben-Hamu, Shamir, Lipman, Invariant and equivariant graph networks, 2019
- [4] Maron, Ben-Hamu, Serviansky, Lipman, Provably powerful graph networks, 2019
- [5] Chen, Villar, Chen, Bruna, On the equivalence between graph isomorphism testing and function approximation with gnns, 2019
- [6] Azizian, Lelarge, Expressive power of invariant and equivariant graph neural networks, 2020
- [7] Dwivedi, Joshi, Laurent, Bengio, Bresson, Benchmarking graph neural networks, 2020

Outline

- Review of graph network architectures
- **Graph positional encoding**
- Transformers for sequences
- Graph Transformers
- ViT/MLP-Mixer for images
- Graph ViT/MLP-Mixer
- Conclusion

Higher expressivity power

- How to improve the limited expressivity of MP-GNNs ?

Intractable

Consider higher-order node interactions (k-tuples with $k > 2$) with WL-GNNs^[1,2,3,4].

- Theorem^[5] (universal approximator) : Any continuous function invariant by permutation can be arbitrarily approximated by WL-GNNs, with the necessary condition the network has higher-order tensor of order $k = \text{poly}(N) = N(N-1)/2$.

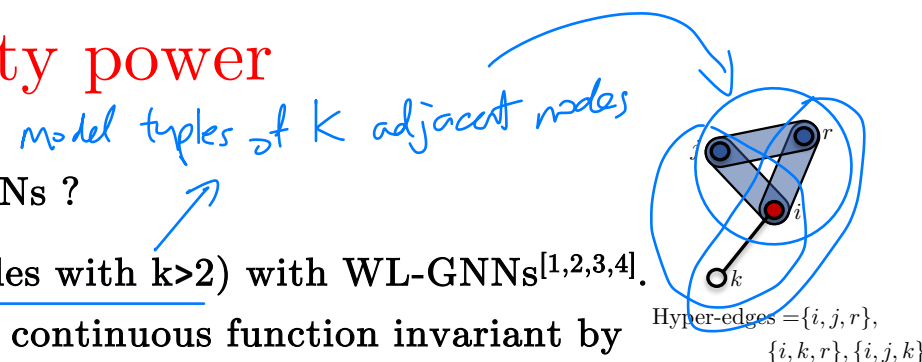
- These networks are computationally expensive, at least $O(N^3)$ for 3-WL expressivity.

Provide a unique ID for each node, i.e. positional encoding (PE).

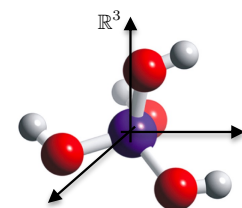
- Theorem^[6] : MP-GNNs are provable more expressive than the 1-WL test when considering node positional encoding.
- Theorem^[7] : MP-GNNs are Turing-complete when depth $d \geq \delta_G$ layers (graph diameter), width is unbounded, and each node is uniquely identified.

- [1] Maron, Ben-Hamu, Shamir, Lipman, Invariant and equivariant graph networks, 2019
- [2] Maron, Ben-Hamu, Serviansky, Lipman, Provably powerful graph networks, 2019
- [3] Chen, Chen, Villar, Bruna, Can graph neural networks count substructures? 2020
- [4] Azizian, Lelarge, Expressive power of invariant and equivariant graph neural networks, 2020
- [5] Maron Fetaya, Segol, Lipman, On the universality of invariant networks, 2019
- [6] Murphy, Srinivasan, Rao, Ribeiro, Relational pooling for graph representations, 2019
- [7] Loukas, What graph neural networks cannot learn: depth vs width, 2019

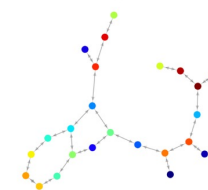
Other positional encoding schemes:
1) Proteins: Angle b/w atoms



Boosts the expressivity a lot (think positional encoding for layers in U-Net SS340)



Extrinsic PE
(e.g. 3D coordinates)



Structural PE
(intrinsic)

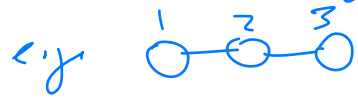
positional encoding candidate

But theory does show that good PE will greatly boost expressivity

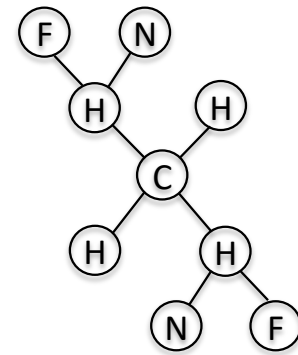
Which structural positional encoding

- Theory does not provide guidance on the choice of structural PE for the class of graphs and task.
- What are properties for designing meaningful PEs?
 - **Uniqueness of representation** for each node
 - **Permutation-invariance**
 - **Distance-sensitivity**: Nodes far apart on the graph should have different positional features whereas nodes nearby have similar positional features.

→ If we change the graph indexing, the proposed PE scheme should still assign the same PE to the same node



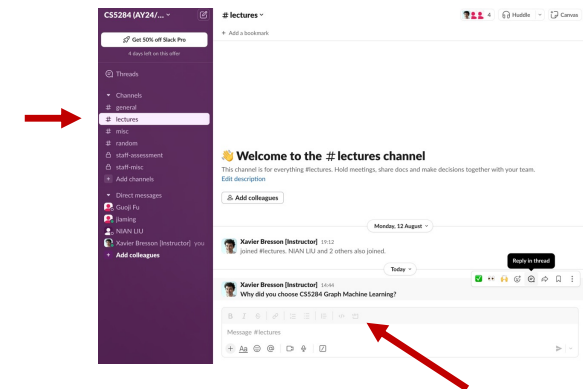
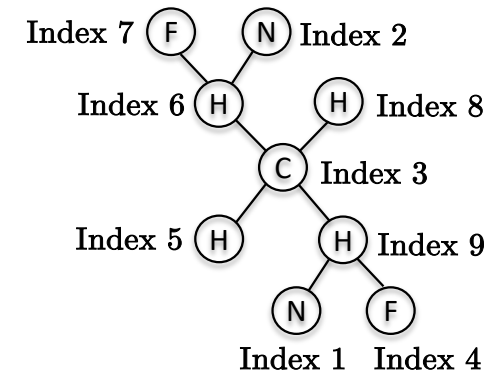
$$A_1 = \frac{1}{3} \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad A_2 = \frac{2}{3} \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$



Regardless if we use A_1 or A_2 to represent the graph, the PE of all nodes should remain consistent

In-lecture question

- The simplest PE is an indexing of the nodes^[1].
 - How many possible indexing exist for a graph of N nodes?
 - How can the network learn different indexing of the graph?
- In Slack #lectures
 - Identify the question and Reply in thread with a short response
- Answer :



[1] Murphy, Srinivasan, Rao, Ribeiro, Relational pooling for graph representations, 2019

Best in General with no domain info

Laplacian Positional Encoding Spectral Embedding

- Laplacian eigenvectors^[1] :
 - Spectral techniques that embed graphs into an Euclidean space.
 - Form a **meaningful local coordinate system**, while preserving the global graph structure.
 - Define via factorization (EVD) of the graph Laplacian matrix.
 - Computational complexity is $O(E^{3/2})$ and $O(N)$ with approximate Nystrom method^[2].
- Laplacian eigenvectors as PE^[3] :
 - Hybrid positional and structural encodings (**invariant by index permutation**).
 - **Unique** and **distance-sensitive** : two nodes far away on graph have large PE distance, and inversely.
 - LapPE are graph generalizations of Transformer's PE^[4].

↳ isn't EVD $O(n^3)$? Different if we want only first k eigenvectors

Qn: For such a graph, how can I have index invariance

$$\Delta = I - D^{-1/2} A D^{-1/2} = U^T \Lambda U$$

Graph Laplacian Degree matrix Adjacency matrix Eigenvalue matrix (Laplacian) Eigenvector matrix

reorder eigenvectors (columns of U) w/ eigenvalues size $u_1, u_2, \dots, u_n$
 If we want a k -dim embedding for each node, construct matrix E

$$E = \begin{bmatrix} | & | & | & | \\ u_1 & u_2 & \dots & u_k \\ | & | & | & | \end{bmatrix}$$

embedding of node 1
embedding of node i

$\lambda_1 < \lambda_2 < \dots < \lambda_k$

- [1] Belkin, Niyogi, Laplacian Eigenmaps and Spectral Techniques for Embedding and Clustering, 2001
 [2] Fowlkes, Belongie, Chung, Malik, Spectral grouping using the nystrom method, 2004
 [3] Dwivedi, Joshi, Laurent, Bengio, Bresson, Benchmarking graph neural networks, 2020
 [4] Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin, Attention is all you need, 2017

$$\Delta = U^T \Lambda U = (-U)^T \Lambda (-U)$$

Limitation of Laplacian PE

sometimes embedding might be diff by -1

- LapPE have natural symmetries with the arbitrary sign of eigenvectors (after being normalized to have unit length).
 → But in practice, we set $K \leq 10$ only
- The number of possible sign flips is 2^K , k being the number of eigenvectors.
- In practice, we choose $K \ll N$, and therefore 2^K is much smaller $N!$ (the number of possible ordering of the nodes). Smaller sampling space than index PEs, and therefore smaller amount of ambiguities to be resolved by the network.
- During the training, eigenvectors will be uniformly sampled at random between the 2^K possibilities^[1].
- Despite this limitation, LapPE have good performance in practice^[1].
- LapPE are used in GraphTransformer^[2], Directed-GNN^[3], SAN^[4].

During training, we randomly flip the signs of eigenvectors to make the NN invariant to the sign.

[1] Dwivedi, Joshi, Laurent, Bengio, Bresson, Benchmarking graph neural networks, 2020

[2] Dwivedi, Bresson, A generalization of transformer networks to graphs, 2020

[3] Beani, Passaro, Letourneau, Hamilton, Corso, Lio, Directional graph networks, 2020

[4] Kreuzer, Beani, Hamilton, Letourneau, Tossou, Rethinking Graph Transformers with Spectral Attention, 2021

Random Walk Positional Encoding

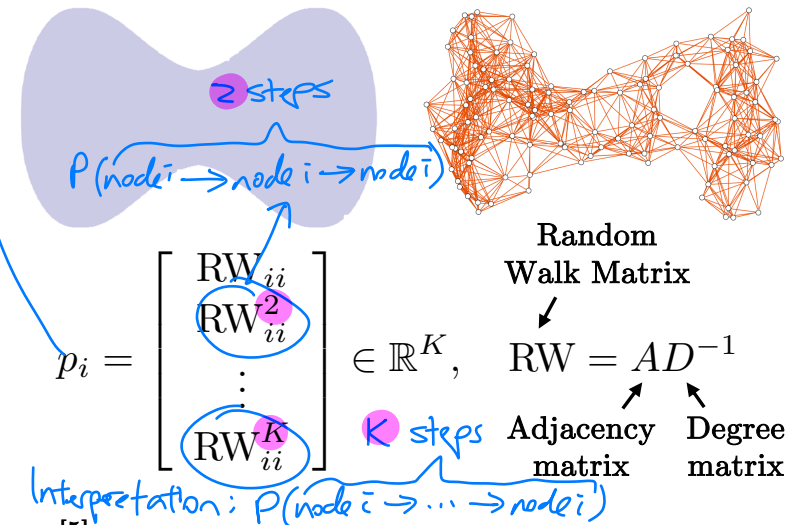
- Diffusion-based PE :

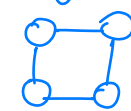
- Diffusion techniques capture graph geometry.
- Popular graph diffusion processes :
 - Diffusion kernel^[2,3], Random walk s.a. pageRank^[4]
- Computational complexity is $O(N^2)$

- Diagonal of K-step of RW matrix as PE^[1] :

- Memory complexity is $O(N)$.
- RWPE is **not limited with the sign ambiguity** like LapPE^[5], i.e. the network is not required to learn additional invariance.
- RWPE **does not guarantee unique** node ID and **distance-sensitive** (no manifold assumption^[6]).
 - Uniqueness of node representation is under the condition that each node has a unique K-hop topological neighborhood for a sufficient large K.

Encodes the geometric structure of the graph with each node



eg.

 All nodes will have same PE

- [1] Dwivedi, Luu, Laurent, Bengio, Bresson, Graph neural networks with learnable structural and positional representations, 2021
- [2] Kondor, Vert, Diffusion kernels, 2004
- [3] Talmon, Cohen, Gannot, Coifman, Diffusion maps for signal processing, 2013
- [4] Page, Brin, Motwani, Winograd, The PageRank citation ranking: Bringing order to the web, 1999
- [5] Dwivedi, Joshi, Laurent, Bengio, Bresson, Benchmarking graph neural networks, 2020
- [6] Belkin, Niyogi, Laplacian Eigenmaps and Spectral Techniques for Embedding and Clustering, 2001

No rule-based PE assignment, learn it.

Learning structural and positional encodings

- Standard MP-GNN inject the positional information into the input layer of the GNN^[1]:

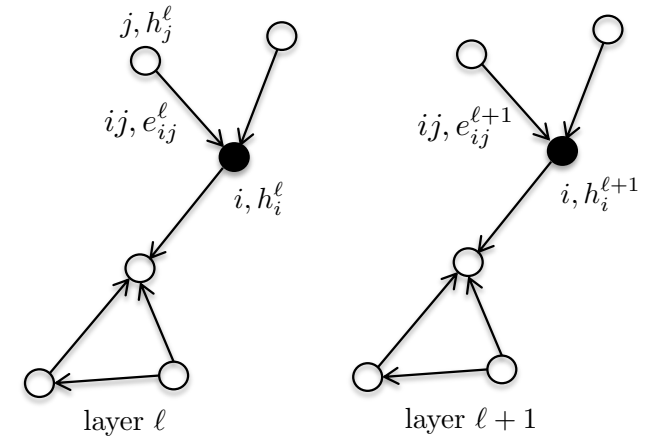
$$\text{Node-update : } h_i^{\ell+1} = f_h(h_i^\ell, \{h_j^\ell\}_{j \in \mathcal{N}_i}, e_{ij}^\ell)$$

$$\text{Edge-update : } e_{ij}^{\ell+1} = f_e(h_i^\ell, h_j^\ell, e_{ij}^\ell)$$

$$\text{with } h_i^{\ell=0} = \text{LE}(\text{Concat}(h_i^0, p_i))$$

graph PE

Input node feature



- We can decouple and learn structural and positional encodings separately^[2]:

$$\text{Node-update : } h_i^{\ell+1} = f_h(h_i^\ell, p_i^\ell, \{h_j^\ell, p_j^\ell\}_{j \in \mathcal{N}_i}, e_{ij}^\ell)$$

$$\text{Edge-update : } e_{ij}^{\ell+1} = f_e(h_i^\ell, h_j^\ell, e_{ij}^\ell)$$

$$\text{PE-update : } p_i^{\ell+1} = f_p(p_i^\ell, \{p_j^\ell\}_{j \in \mathcal{N}_i})$$

PE of neighbours of central node i

[1] Dwivedi, Joshi, Laurent, Bengio, Bresson, Benchmarking graph neural networks, 2020

[2] Dwivedi, Luu, Laurent, Bengio, Bresson, Graph neural networks with learnable structural and positional representations, 2021

GatedGCNs with learnable PE

- **Augmented architecture**^[1,2] :

$$h^{\ell+1}, e^{\ell+1}, \textcircled{p_i^{\ell+1}} = \text{GatedGCN-PE}(h^\ell, e^\ell, p^\ell), \quad h \in \mathbb{R}^{n \times d}, e \in \mathbb{R}^{E \times d}, p \in \mathbb{R}^{n \times K}$$

defined as

$$h_i^{\ell+1} = h_i^\ell + \text{ReLU}\left(\text{BN}\left(A_1^\ell \begin{bmatrix} h_j^\ell \\ \textcircled{p_i^\ell} \end{bmatrix} + \sum_{j \in \mathcal{N}_i} \eta_{ij}^\ell \odot A_2^\ell \begin{bmatrix} h_j^\ell \\ \textcircled{p_j^\ell} \end{bmatrix}\right)\right)$$

$$e_{ij}^{\ell+1} = e_{ij}^\ell + \text{ReLU}\left(\text{BN}\left(B_1^\ell h_i^\ell + B_2^\ell h_j^\ell + B_3^\ell e_{ij}^\ell\right)\right) \quad \text{Positional representation that augments any MP-GNN.}$$

$$\eta_{ij}^\ell = \frac{\sigma(e_{ij}^\ell)}{\sum_{j' \in \mathcal{N}_i} \sigma(e_{ij'}^\ell) + \varepsilon}$$

$$\textcircled{p_i^{\ell+1}} = p_i^\ell + \tanh\left(C_1^\ell p_i^\ell + \sum_{j \in \mathcal{N}_i} \eta_{ij}^\ell \odot C_2^\ell p_j^\ell\right) \quad \text{with} \quad \text{mean}(p_{\cdot, k}^{\ell+1}) = 0, \quad \|p_{\cdot, k}^{\ell+1}\| = 1, \quad \forall k$$

[1] Bresson, Laurent, Residual gated graph convnets, 2017

[2] Dwivedi, Luu, Laurent, Bengio, Bresson, Graph neural networks with learnable structural and positional representations, 2021

Positional encoding loss

- Positional loss can be considered.
 - Laplacian eigenvector loss^[1,2] can be used to enforce the PE to form a coordinate system constrained by the graph topology.
- Final loss is composed of two terms :

$$L = L_{\text{Task}}\left(\begin{bmatrix} h^L \\ p^L \end{bmatrix}\right) + \alpha L_{\text{LapEig}}(p^L)$$

$$L_{\text{LapEig}}(p) = \text{trace}(p^T \Delta p) + \lambda \|p^T p - \mathbf{I}_K\|_{\text{F}}^2$$

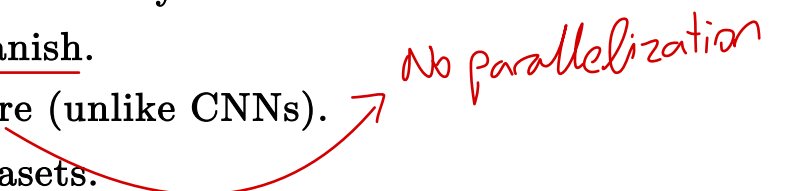
[1] Belkin, Niyogi, Laplacian Eigenmaps and Spectral Techniques for Embedding and Clustering, 2001

[2] Lai, Osher, A splitting method for orthogonality constrained problems, 2014

Outline

- Review of graph network architectures
- Graph positional encoding
- **Transformers for sequences**
- Graph Transformers
- ViT/MLP-Mixer for images
- Graph ViT/MLP-Mixer
- Conclusion

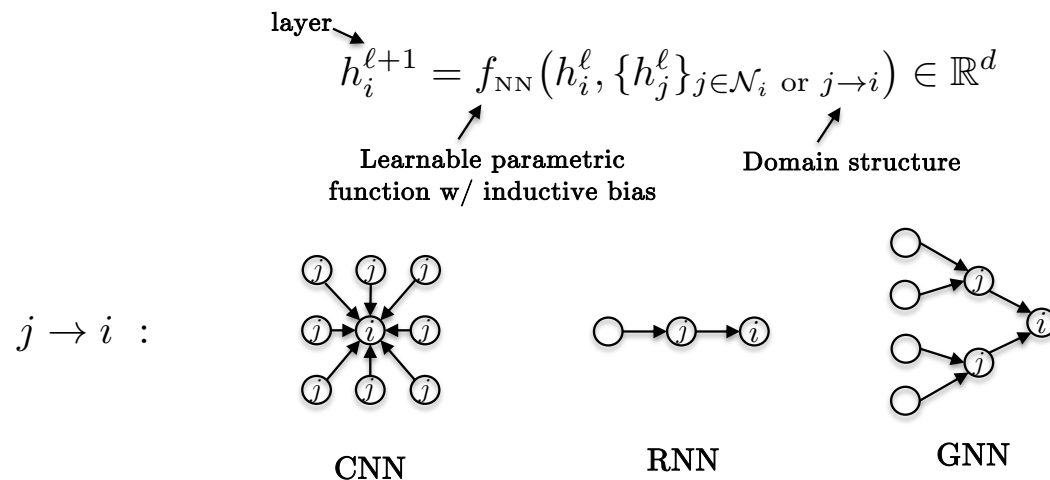
RNNs for sequences

- RNNs like LSTM^[1] are designed for sequences.
- Contributions
 - They learn representation of sequences independently of their length using a recurrence formula with weight sharing across time (translation invariance).
 - They process ordered sequences, i.e. 1D regular lattice, by design.
- Limitations
 - **Hard to train** because they are non-linear dynamical systems.
 - Any small perturbation can amplify or vanish.
 - **Slow to train** because of their sequential nature (unlike CNNs).  *No parallelization*
 - Important limitation with large-scale datasets.
 - RNNs cannot learn **long-term dependencies** (no more than 50 steps) between words because of **over-squashing**.

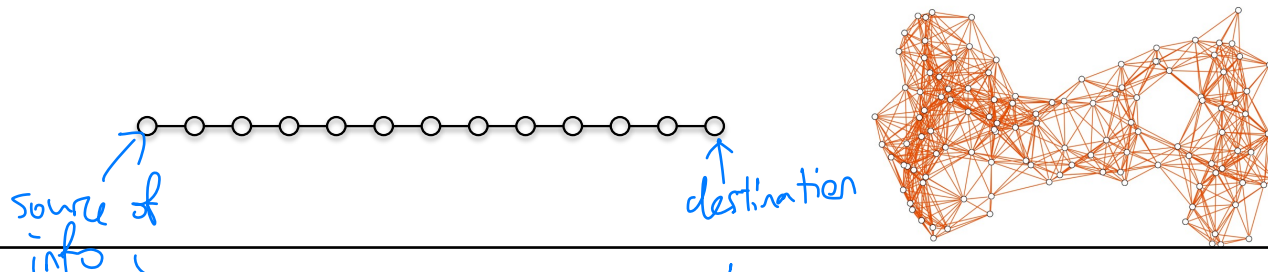
[1] Hochreiter, Schmidhuber, Long short-term memory, 1997 (114,000 citations)

Over-squashing

- Most neural networks s.a. CNNs, RNNs, GNNs are **message-passing techniques**.
 - New representation is computed by aggregating (with f_{NN}) the neighborhood information $j \rightarrow i$:



- As the MP mechanism is local for CNNs/RNNs/GNNs, they **require to stack multiple layers to propagate information** over long-range distances (ideally over the whole sequence/graph).



need this amount of layers stacked (if each layer can only propagate info to its immediate neighbours, which is desirable to build the hierarchy of features).

Over-squashing

Too many "noisy" intermediate vectors, and only a few vectors are relevant to update h_i^{l+1}

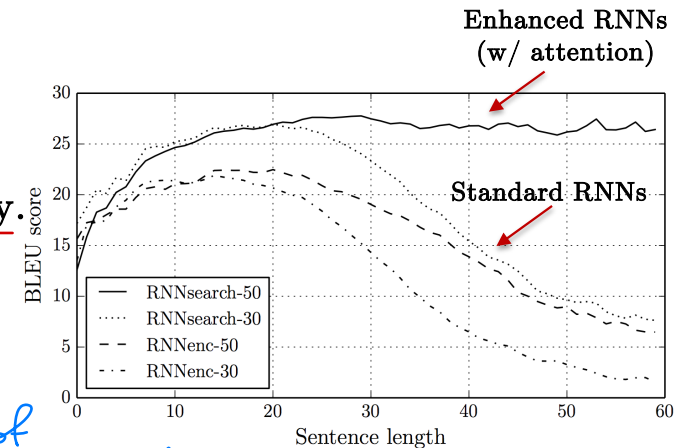
- Stacking layers to propagate information though the domain can be a major issue.
 - The reception field grows as $O(L^2)$ for CNNs and $O(L)$ for RNNs, L being the number of layers.
- The reception field size gives the number of neighbors used to update the representation.
 - For RNNs, this requires aggregating $O(L)$ number of vectors:

$$h_i^{\ell+1} = F_{\text{RNN}}(h_i^0, h_{i-1}^0, h_{i-2}^0, \dots, h_{i-L}^0) \in \mathbb{R}^d$$

receptive field of node i in layer $\ell+1$

- This issue is known as **over-squashing**.
 - NNs w/ local reception field cannot learn long-range dependency.
 - Vanilla RNNs cannot process sequences more than 20 words^[1].
 - This means that MP-GNNs cannot stack many layers.

previous layer ℓ will be a function of $L-1$ input nodes (with larger emphasis on input nodes ℓ as older input nodes decay in importance)

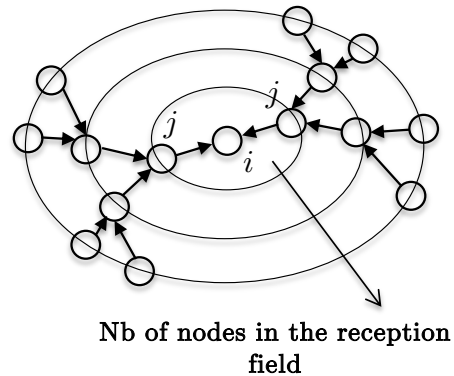


Figure^[1]. Accuracy of translated sequences (higher is better).

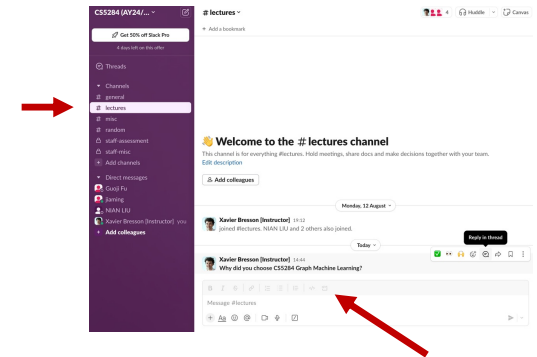
[1] Bahdanau, Cho, Bengio, Neural machine translation by jointly learning to align and translate, 2014 (36,000 citations)

In-lecture question

- How does the reception field grow w.r.t. the number L of layers for tree graphs?
- In Slack #lectures
 - Identify the question and Reply in thread with a short response
- Answer :



$$h_i^{\ell+1} = F_{\text{GNN}}(\dots) \in \mathbb{R}^d$$

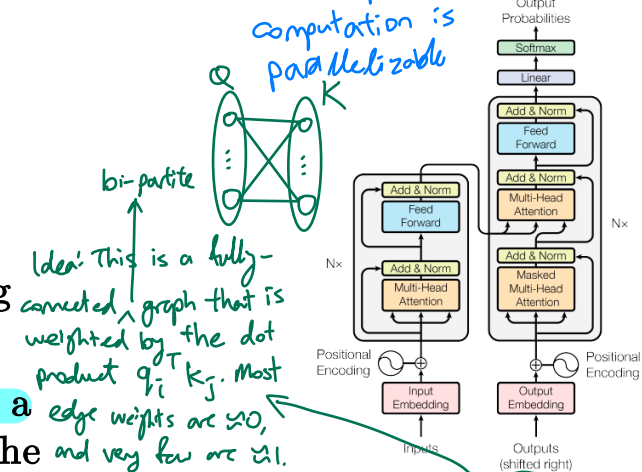
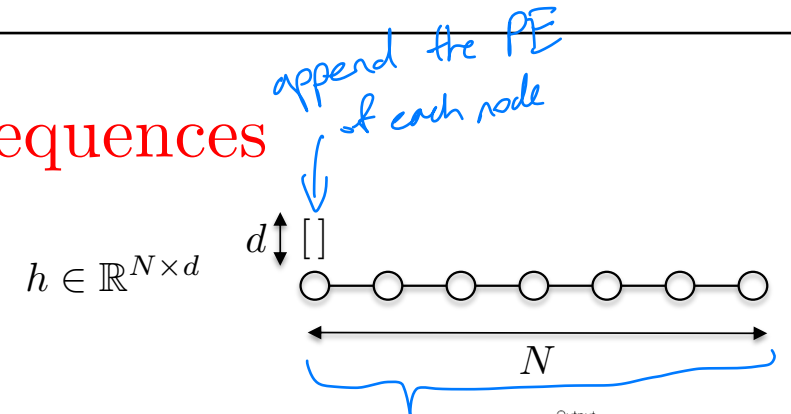


Transformers for sequences

- Key ingredients of original Transformers^[1]
 - Attention mechanism (self/cross attention)
 - Layer normalization + residual connection + MLP
 - Cos/sin functions as node positional encoding

- Contributions

- Attention layers **fixes the over-squashing** issue by connecting all words together with pair-wise matching (attention scores).
- Attention layers produce a **dynamic representation of a word depending on the context** (e.g. the vase **broke**, the news **broke**, Sandy **broke** the law, etc) $\neq$ word2vec.



forces sparsity by only focusing on a few other nodes in the NN

$$h^{\ell+1} = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \in \mathbb{R}^{N \times d}$$

$$Q = h^\ell W_Q^\ell \in \mathbb{R}^{N \times d}$$

$$K = h^\ell W_K^\ell \in \mathbb{R}^{N \times d}$$

$$V = h^\ell W_V^\ell \in \mathbb{R}^{N \times d}$$

[1] Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin, Attention is all you need, 2017

Even though reception field in worst case can grow by $O(N)$, on average, each node only focuses on $k \ll N$ nodes in the next layer

see prev slide note

Limitations of Transformers

- Limitations :

- They fix the issues of over-squashing and long-range dependency by “connecting everything to everything” but the attention mechanism is costly :

- The reception field is the size of the domain $O(N)$.

- Memory/speed complexity is $O(N^2d)$. *slow*

- This limits Transformers on GPUs to only process **short sequences**, $N \leq 1,000$ for $d=512$.

- They cannot process ordered sequences, i.e. 1D regular lattice.

- Transformers are **set** neural networks – they are designed to process **sets of data** points.

- They require additional data structure s.a. **cos/sin functions as node positional encoding** to represent explicitly the position of the words in the sequence.

the matrix multiplication.
 QK^T is $O(n^2)$ in memory & time to compute. It's also a FULL matrix
↳ otherwise have memory issue

Outline

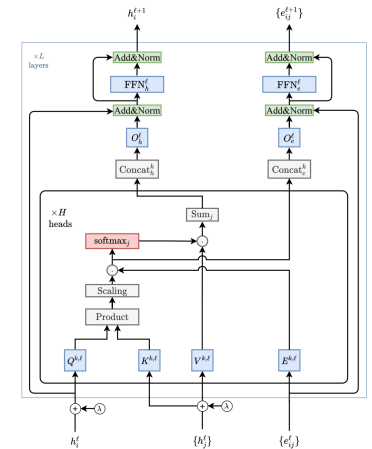
- Review of graph network architectures
- Graph positional encoding
- Transformers for sequences
- **Graph Transformers**
- ViT/MLP-Mixer for images
- Graph ViT/MLP-Mixer
- Conclusion

too many features $\rightarrow$ Not enough data
 $\therefore$ We need some way to intelligently
 reduce/restrict our model's expressivity.

$\hookrightarrow$ Use a graph to represent our domain
 knowledge/prior to reduce the
 # of parameters our model
 must learn

Graph Transformers

- Contributions of GT^[1]
 - Generalizing Transformers^[2,3] to arbitrary graphs with $O(E+N)$ complexity.
 - Leveraging graph as topological inductive bias.
 - Generalizing cos/sin positional encoding (PE) to graphs (node ordering) with Laplacian eigenvectors.
 - Introducing edge feature through bilinear product (for e.g. molecular bond or relationship between entities in knowledge graph).
 - Graph Transformer (GT) and its improvements^[4] have gained popularity as a backbone architecture in biology (used by winner and runner-up at NeurIPS'22 OGB Large-Scale Challenge competition^[5])
- Limitations
 - Over-squashing and poor long-range dependency
 - Small graphs $\leftarrow$ only works well with small graphs.



- [1] Dwivedi, Bresson, A generalization of transformer networks to graphs, AAAI 2021
- [2] Bahdanau, Cho, Bengio, Neural machine translation by jointly learning to align and translate, 2014
- [3] Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin, Attention is all you need, 2017
- [4] Ying, Cai, Luo, Zheng, Ke, He, Shen, Liu, Do Transformers Really Perform Bad for Graph Representation, 2021
- [5] <https://ogb.stanford.edu/neurips2022/workshop>

Transformers

- Standard Transformer layer (for sequences^[1] w/PE, grids/images^[2], sets wo/PE) :

$$\begin{aligned} \bar{h}^\ell &= h^\ell + \text{MHA}(\text{LN}(h^\ell)) \in \mathbb{R}^{N \times d} \\ h^{\ell+1} &= \bar{h}^\ell + \text{MLP}(\text{LN}(\bar{h}^\ell)) \in \mathbb{R}^{N \times d} \end{aligned}$$

layer normalization seq length / #pixels / #set elements

with $\text{MHA}(h) = \text{Concat}_{k=1}^H (\text{HA}(h_k)) W_O \in \mathbb{R}^{N \times d}$, $h_k \in \mathbb{R}^{N \times d' = d/H}$, $W_O \in \mathbb{R}^{d \times d}$

Multi-head attention Head attention matching score

$$\text{HA}(h) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d'}} \right) V \in \mathbb{R}^{N \times d' = d/H}$$

$$\text{HA}(h)_i = \sum_{j \in N} \underbrace{\frac{\exp(q_i^T k_j / \sqrt{d'})}{\sum_{j' \in N} \exp(q_i^T k_{j'} / \sqrt{d'})}}_{\text{attention score}_{ij}} v_j \quad (\text{point-wise equation})$$

Q: query, K: key, V: value

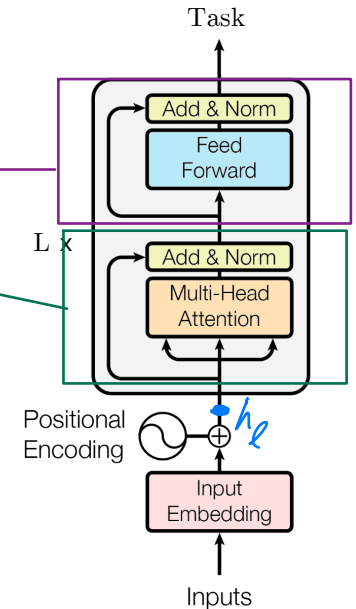
naïve token embedding positional encoding

$$h^{\ell=0} = \text{LL}(h_0) + p_0 \in \mathbb{R}^{N \times d} \quad (\text{input word/patch and positional encoding})$$

$$Q = h_k W_Q, K = h_k W_K, V = h_k W_V \in \mathbb{R}^{N \times d' = d/H}, W_Q, W_K, W_V \in \mathbb{R}^{d' \times d'}$$

cos/sin PE

$$\text{with } (p_0)_{ik} = \begin{cases} \sin(2\pi f_k i) & \text{if } i \text{ even} \\ \cos(2\pi f_k i) & \text{if } i \text{ odd} \end{cases} \quad \text{with } f_k = \frac{10,000^{\frac{d}{[2k]}}}{2\pi} \quad \parallel \text{cos/sin positional encoding scheme}$$



- [1] Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin, Attention is all you need, 2017
 [2] Dosovitskiy et-al, An image is worth 16x16 words: Transformers for image recognition at scale, 2020

Graph Transformers

- Graph Transformer layer^[1] (for arbitrary graph) :

$$\begin{aligned} \bar{h}^\ell &= h^\ell + \text{gMHA}(\text{LN}(h^\ell)) \in \mathbb{R}^{N \times d} \\ h^{\ell+1} &= \bar{h}^\ell + \text{MLP}(\text{LN}(\bar{h}^\ell)) \in \mathbb{R}^{N \times d} \end{aligned}$$

nodes

same as before, except
mHA replaced by gMHA

only allows v_i to have a connection with v_j if $(v_i, v_j) \in \text{Edges}$

Graph multi-head attention

$$\text{gHA}(h) = \text{Softmax} \left(A_G \odot \frac{QK^T}{\sqrt{d'}} \right) V \in \mathbb{R}^{N \times d' = d/H},$$

Graph head attention

$$A_G \in \mathbb{R}^{N \times N} \text{ (graph adjacency matrix)}$$

$$\text{gHA}(h)_i = \sum_{j \in \mathcal{N}_i} \underbrace{\frac{\exp(q_i^T k_j / \sqrt{d'})}{\sum_{j' \in \mathcal{N}_i} \exp(q_i^T k_{j'} / \sqrt{d'})}}_{\text{graph attention score}_{ij}} v_j \quad (\text{point-wise equation})$$

neighbors of node i

$$Q = h_k W_Q, K = h_k W_K, V = h_k W_V \in \mathbb{R}^{N \times d' = d/H}, W_Q, W_K, W_V \in \mathbb{R}^{d' \times d'}$$

$$h^{\ell=0} = \text{LL}_1(h_0) + \text{LL}_2(p_0) \in \mathbb{R}^{N \times d} \text{ (input node feature and positional encoding)}$$

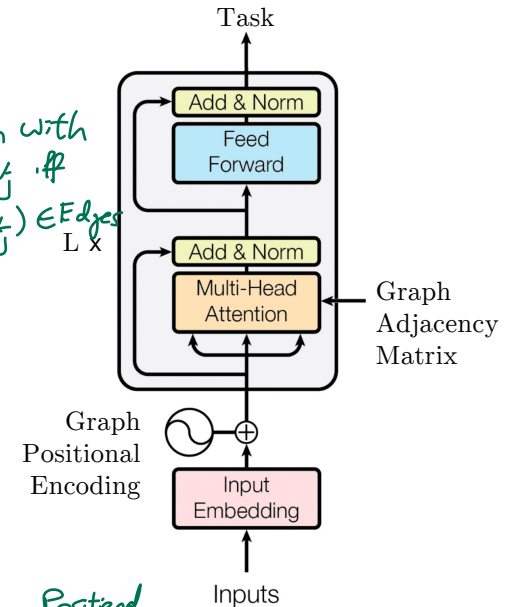
$$\text{with } p_0 = \Phi_{\{2, \dots, K+1\}} \in \mathbb{R}^{N \times K}, \Delta = \Phi \Lambda \Phi^T \in \mathbb{R}^{N \times N}$$

First non-trivial K eigenvectors

Laplacian

Eigenvectors / eigenvalues

Laplacian Positional Encoding scheme.



[1] Dwivedi, Bresson, A generalization of transformer networks to graphs, 2020

Graph adjacency matrix

- Definition of adjacency matrix A_G in the graph head attention function :

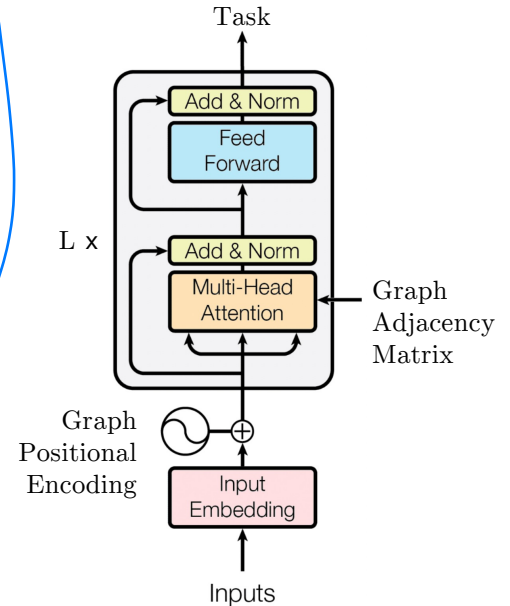
$$\text{gHA}(h) = \text{Softmax} \left(A_G \odot \frac{QK^T}{\sqrt{d'}} \right) V \in \mathbb{R}^{N \times d'}$$

$$\text{with } A_{Gij} = \begin{cases} A_{ij}^{\text{original}} & \text{if } ij \text{ connected} \\ 1 & \text{for } j = i \text{ (enforcing self-loop)} \\ -\infty & \text{if } ij \text{ not connected} \end{cases}$$

so that $\exp(-\infty) = 0$

- Alternative formulation :

$$\begin{aligned} \text{gHA}(h) &= \text{Normalize}_{\text{dim=rows}} \left(A'_G \odot \exp \left(\frac{QK^T}{\sqrt{d'}} \right) \right) V \in \mathbb{R}^{N \times d'} \\ &= A'_G \odot \exp \left(\frac{QK^T}{\sqrt{d'}} \right) \left[\left(A'_G \odot \exp \left(\frac{QK^T}{\sqrt{d'}} \right) \right) \mathbf{1}_{N \times N} \right]^{-1} V \in \mathbb{R}^{N \times d'} \\ \text{with } A'_{Gij} &= \begin{cases} A_{ij}^{\text{original}} & \text{if } ij \text{ connected} \\ 1 & \text{for } j = i \text{ (enforcing self-loop)} \end{cases} \end{aligned}$$



$\begin{cases} 1 & \text{if } ij \text{ connected or } i=j \\ -\infty & \text{otherwise} \end{cases}$

same performance as GAT

'Like any BNN/GCN like GAT' (?)

Lab 1 : Vanilla GT

No positional encoding

just uses $gHA(h) = \text{softmax}\left(A_G \odot \frac{QK^T}{\mathbb{I}}\right)V$

• GT without graph PE

Lecture : Graph Transformers & Graph ViT

Lab 01 : Vanilla Graph Transformers (GT without Positional Encoding)

Xavier Bresson

Dwivedi, Bresson, A generalization of transformer networks to graphs, 2020
<https://arxiv.org/pdf/2012.09699.pdf>

```
# For Google Colaboratory
import sys, os
if 'google.colab' in sys.modules:
    # mount google drive
    from google.colab import drive
    drive.mount('/content/gdrive')
    path_to_file = '/content/gdrive/My Drive/CS5284_2024_codes/codes/10_Graph_Transformers'
    print(path_to_file)
    # change current path to the folder containing "path_to_file"
    os.chdir(path_to_file)
    !pwd
    !pip install dgl==1.0.0 # Install DGL
```

```
# Libraries
import dgl
from dgl.data import MiniGCDataset
import dgl.function as fn
import matplotlib.pyplot as plt
import networkx as nx
import torch
from torch.utils.data import DataLoader
import torch.nn as nn
import time
```

Design the class of vanilla GraphTransformer networks with DGL

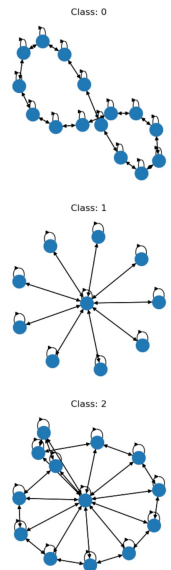
Node update equation:

$$\begin{aligned} \bar{h}^t &= h^t + \text{gMHA}(\text{LN}(h^t)) \in \mathbb{R}^{N \times d} \\ h^{t+1} &= \bar{h}^t + \text{MLP}(\text{LN}(\bar{h}^t)) \in \mathbb{R}^{N \times d} \\ \text{with } \text{gMHA}(h) &= \text{Concat}_{k=1}^H (\text{gHA}(h_k)) W_O \in \mathbb{R}^{N \times d}, h_k \in \mathbb{R}^{N \times d=H}, W_O \in \mathbb{R}^{d \times d} \\ \text{gHA}(h) &= \text{Softmax}\left(A_G \odot \frac{QK^T}{\sqrt{d'}}\right)V \in \mathbb{R}^{N \times d=H}, A_G \in \mathbb{R}^{N \times N} \text{ (graph adjacency matrix)} \\ \text{gHA}(h)_i &= \sum_{j \in N_i} \frac{\exp(q_i^T k_j / \sqrt{d'})}{\sum_{j \in N_i} \exp(q_i^T k_j / \sqrt{d'})} v_j \text{ (point-wise equation)} \\ &\quad \text{graph attention score}_{ij} \\ Q &= h_k W_Q, K = h_k W_K, V = h_k W_V \in \mathbb{R}^{N \times d=H}, W_Q, W_K, W_V \in \mathbb{R}^{d' \times d} \\ h^{t=0} &= \text{LL}(h_0) \in \mathbb{R}^{N \times d} \text{ (input node feature)} \end{aligned}$$

```
# class MLP layer for classification
class MLP_Layer(nn.Module):

    def __init__(self, input_dim, output_dim, L=2): # L = nb of hidden layers
        super(MLP_Layer, self).__init__()
        list_FC_layers = [ nn.Linear( input_dim, input_dim, bias=True ) for l in range(L) ]
        list_FC_layers.append( nn.Linear( input_dim, output_dim, bias=True ) )
        self.FC_layers = nn.ModuleList(list_FC_layers)
        self.L = L

    def forward(self, x):
        y = x
        for l in range(self.L):
            y = self.FC_layers[l](y)
            y = torch.relu(y)
        y = self.FC_layers[self.L](y)
        return y
```



100% necessary for this task

Lab 2 : Graph Transformers

← Includes positional encoding

● GT with graph Laplacian PE

Lecture : Graph Transformers & Graph ViT

Lab 02 : Graph Transformers (GT with Positional Encoding)

Xavier Bresson

Dwivedi, Bresson, A generalization of transformer networks to graphs, 2020
<https://arxiv.org/pdf/2012.09699.pdf>

```
# For Google Colaboratory
import sys, os
if 'google.colab' in sys.modules:
    # mount google drive
    from google.colab import drive
    drive.mount('/content/gdrive')
    path_to_file = '/content/gdrive/My Drive/CS5284_2024_codes/codes/10_Graph_Transformers'
    print(path_to_file)
    # change current path to the folder containing "path_to_file"
    os.chdir(path_to_file)
    !pwd
    !pip install dgl==1.0.0 # Install DGL
```

```
# Libraries
import dgl
from dgl.data import MiniGCDataset
import dgl.function as fn
import matplotlib.pyplot as plt
import networkx as nx
import torch
from torch.utils.data import DataLoader
import torch.nn as nn
import time
import sys; sys.path.insert(0, 'lib/')
from lib.utils import compute_ncut
```

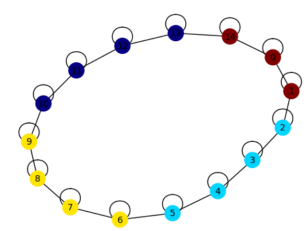
Design the class of GraphTransformer networks (with positional encoding)

Node update equation:

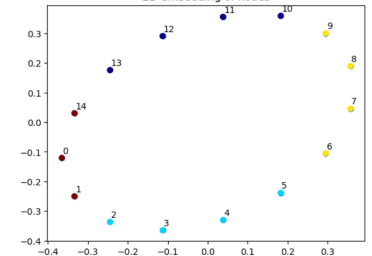
$$\begin{aligned} \tilde{h}^t &= h^t + \text{gMHA}(\text{LN}(h^t)) \in \mathbb{R}^{N \times d} \\ h^{t+1} &= \tilde{h}^t + \text{MLP}(\text{LN}(\tilde{h}^t)) \in \mathbb{R}^{N \times d} \\ \text{with } \text{gMHA}(h) &= \text{Concat}_{i=1}^H (\text{gHA}(h_k)) W_O \in \mathbb{R}^{N \times d}, h_k \in \mathbb{R}^{N \times d-d/H}, W_O \in \mathbb{R}^{d \times d} \\ \text{gHA}(h) &= \text{Softmax} \left(A_G \odot \frac{QK^T}{\sqrt{d^H}} \right) V \in \mathbb{R}^{N \times d-d/H}, A_G \in \mathbb{R}^{N \times N} \text{ (graph adjacency matrix)} \\ \text{gHA}(h)_i &= \sum_{j \in N_i} \frac{\exp(q_i^T k_j / \sqrt{d^H})}{\sum_{j' \in N_i} \exp(q_i^T k_{j'} / \sqrt{d^H})} v_j \text{ (point-wise equation)} \\ &\quad \text{graph attention scores}_{ij} \\ Q &= h_k W_Q, K = h_k W_K, V = h_k W_V \in \mathbb{R}^{N \times d-d/H}, W_Q, W_K, W_V \in \mathbb{R}^{d^H \times d} \\ h^{t=0} &= \text{LL}_1(h_0) + \text{LL}_2(p_0) \in \mathbb{R}^{N \times d} \text{ (input node feature and positional encoding)} \\ \text{with } p_0 &= \Phi_{\{2, \dots, K+1\}} \in \mathbb{R}^{N \times K}, \Delta = \Phi \Lambda \Phi^T \in \mathbb{R}^{N \times N} \end{aligned}$$

```
# class MLP layer for classification
class MLP_layer(nn.Module):
    def __init__(self, input_dim, output_dim, L=2): # L = nb of hidden layers
        super(MLP_layer, self).__init__()
        list_FC_layers = [ nn.Linear( input_dim, input_dim, bias=True ) for l in range(L) ]
        list_FC_layers.append(nn.Linear( input_dim, output_dim, bias=True ))
        self.FC_layers = nn.ModuleList(list_FC_layers)
        self.L = L
    def forward(self, x):
        y = x
        for l in range(self.L):
            y = self.FC_layers[l](y)
            y = torch.relu(y)
        y = self.FC_layers[self.L](y)
        return y
```

Visualization with networkx



2D embedding of nodes



Graph Transformers with edge features

Node feature update^[1]:

$$\begin{aligned}\bar{h}^\ell &= h^\ell + \text{gMHA}(\text{LN}(h^\ell), \text{LN}(e^\ell)) \in \mathbb{R}^{N \times d} \\ h^{\ell+1} &= \bar{h}^\ell + \text{MLP}(\text{LN}(\bar{h}^\ell)) \in \mathbb{R}^{N \times d}\end{aligned}$$

with $\text{gMHA}(h, e) = \text{Concat}_{k=1}^H (\text{gHA}(h_k, e_k)) W_O \in \mathbb{R}^{N \times d}$, $h_k \in \mathbb{R}^{N \times d'} = d/H$, $e_k \in \mathbb{R}^{E \times d'}$, $W_O \in \mathbb{R}^{d \times d}$

$$\text{gHA}(h, e)_i = \sum_{j \in \mathcal{N}_i} \underbrace{\frac{\exp(q_i^T \text{diag}(e_{ij}) k_j / \sqrt{d'})}{\sum_{j' \in \mathcal{N}_i} \exp(q_i^T \text{diag}(e_{ij'}) k_{j'} / \sqrt{d'})}}_{\text{graph attention score w/ edge feature}_{ij}} v_j \quad (\text{point-wise equation})$$

$$Q = h_k W_Q, K = h_k W_K, V = h_k W_V \in \mathbb{R}^{N \times d'} = d/H, E = e_k W_E \in \mathbb{R}^{E \times d'} = d/H, W_Q, W_K, W_V, W_L$$

$$h^{\ell=0} = \text{LL}_1(h_0) + \text{LL}_2(p_0) \in \mathbb{R}^{N \times d} \quad (\text{input node feature and positional encoding})$$

with $p_0 = \Phi_{\{2, \dots, K+1\}} \in \mathbb{R}^{N \times K}$, $\Delta = \Phi \Lambda \Phi^T \in \mathbb{R}^{N \times N}$

Edge feature update^[1]:

$$\begin{aligned}\bar{e}^\ell &= e^\ell + \text{gMHE}(\text{LN}(e^\ell), \text{LN}(h^\ell)) \in \mathbb{R}^{E \times d} \\ e^{\ell+1} &= \bar{e}^\ell + \text{MLP}(\text{LN}(\bar{e}^\ell)) \in \mathbb{R}^{E \times d}\end{aligned}$$

with $\text{gMHE}(e, h) = \text{Concat}_{k=1}^H (\text{gHE}(e_k, h_k)) W_O^e \in \mathbb{R}^{E \times d}$, $h_k \in \mathbb{R}^{N \times d'} = d/H$, $e_k \in \mathbb{R}^{E \times d'}$, $W_O^e \in \mathbb{R}^{d \times d}$

with $\text{gHE}(e, h)_{ij} = q_i \odot e_{ij} \odot k_j / \sqrt{d'} \in \mathbb{R}^{d'}$ (point-wise equation)

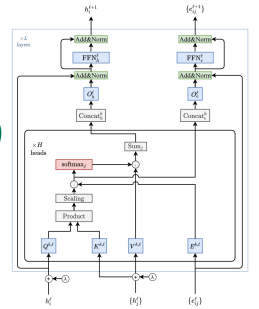
$$e^{\ell=0} = \text{LL}(e_0) \in \mathbb{R}^{E \times d} \quad (\text{input edge feature})$$

Hadamard product w/ node and edge features

Now this encodes graph topology, and is not just 0 or 1 like in here

Bilinear product w/ node and edge features

How are the edge features stored/represented here? Vector of size $|E|$ (?)



As opposed to before
 $\text{gMHA}(h) = \text{Softmax} \left[A_G \odot \frac{QK^T}{\sqrt{d'}} \right] V$
 encodes graph topology

why no h ? h is already used to compute q_i and k_j

[1] Dwivedi, Bresson, A generalization of transformer networks to graphs, 2020

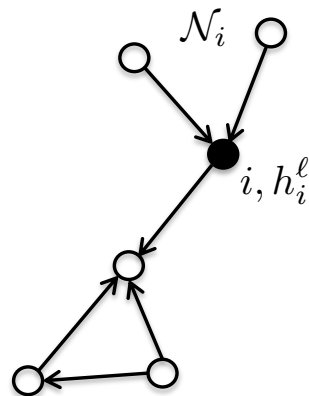
Graph Transformer is just a generalization of normal transformer

Consistency

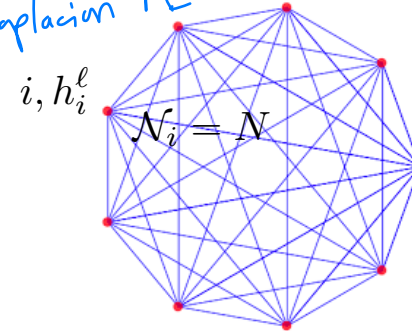
special case

- Graph Transformers^[1] reduces to (original) Transformers^[2] when
 - The graph is fully connected, i.e. $\mathcal{N}_i = N$ (set of all nodes)
 - The value of edge feature is 1, i.e. $e_{ij} = 1_d$ (d -dim vector of ones)
 - The LapPE are cos/sin functions for sequences

No edge features
aka GT with Laplacian PE



Sparsely connected graph/
Graph Transformers^[1]



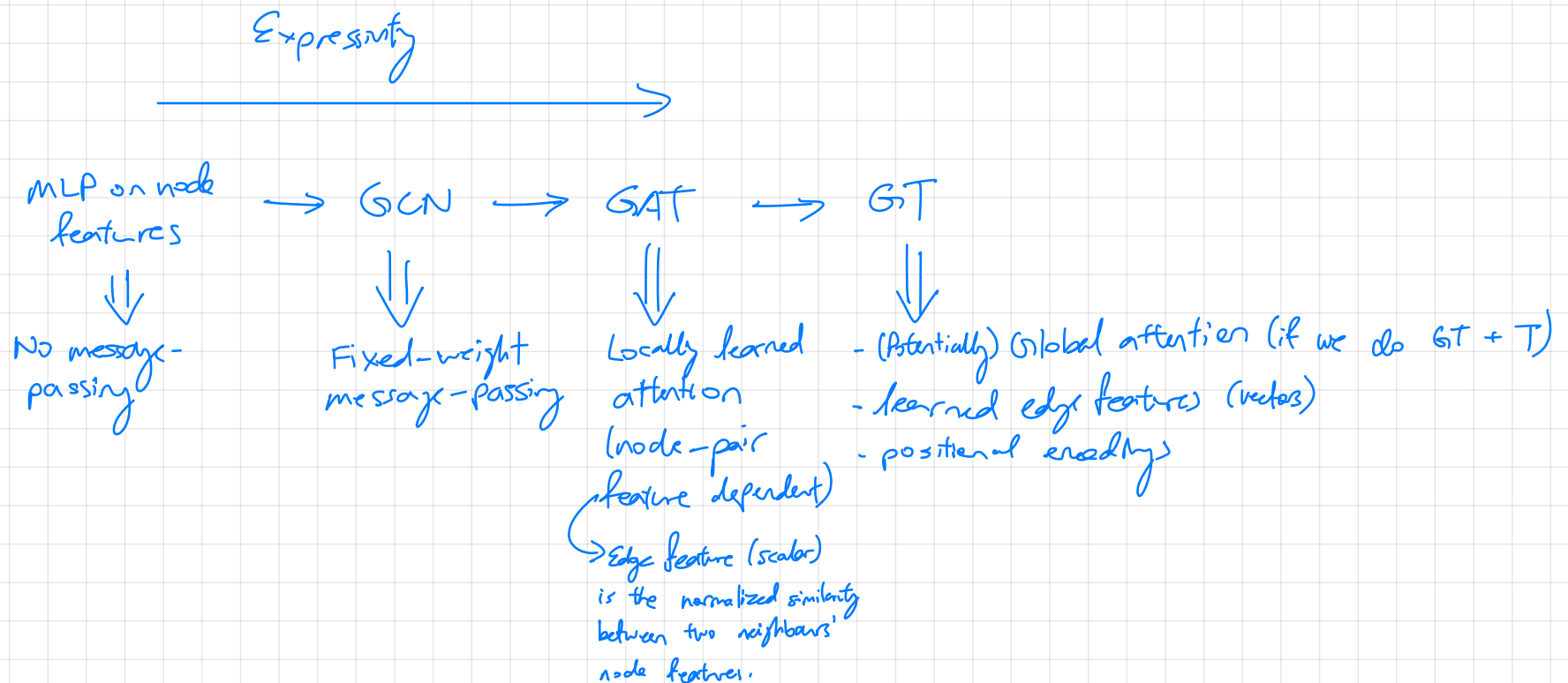
Fully connected graph/
Transformers^[2]

[1] Dwivedi, Bresson, A generalization of transformer networks to graphs, 2020

[2] Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin, Attention is all you need, 2017

Note: GCNs are also a specific kind of GT

- Fixed, uniform attention weights across all neighbours.
- No edge features
- Fixed Q, K matrix, $W_Q = W_K = I_d$
- No positional encoding
- Fixed Adjacency matrix



Best model in biology

Learn the best embedding/
representation of the whole

Lab 3 : GT with edge features

molecule

Downstream task :

• GT with edge features and DGL

(molecules/materials)

Just pass into an MLP || Regression with the graph/molecule embedding (energy of molecule)

Lecture : Graph Transformers & Graph ViT

Lab 03 : Graph Transformers with edge features and DGL (sparse linear algebra)

Xavier Bresson

Dwivedi, Bresson, A generalization of transformer networks to graphs, 2020
<https://arxiv.org/pdf/2012.09699.pdf>

```
# For Google Colaboratory
import sys, os
if 'google.colab' in sys.modules:
    # mount google drive
    from google.colab import drive
    drive.mount('/content/gdrive')
    path_to_file = '/content/gdrive/My Drive/CS5284_2024_codes/codes/10_Graph_Transformers'
    print(path_to_file)
    # change current path to the folder containing "path_to_file"
    os.chdir(path_to_file)
    !pwd
    !pip install dgl # Install DGL
```

```
# Libraries
import pickle
import dgl
import dgl.function as fn
import matplotlib.pyplot as plt
import torch
from torch.utils.data import DataLoader
import torch.nn as nn
import time
import networkx as nx
import sys; sys.path.insert(0, 'lib/')
from lib.utils import compute_ncut
from lib.molecules import Dictionary, MoleculeDataset, MoleculeDGL, Molecule
import os, datetime
```

Large graph

Xavier Bresson

cannot use Pytorch
(very slow ∵ Not optimized
for sparse graphs)

or

Pytorch (not optimized for sparse linear algebra)

Design the class of GraphTransformer networks with edge features

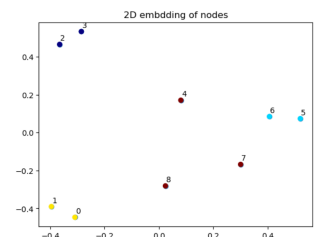
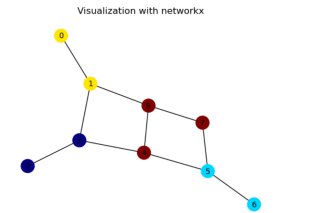
Node update equation:

$$\begin{aligned}\tilde{h}^t &= h^t + \text{gMHA}(\text{LN}(h^t), \text{LN}(e^t)) \in \mathbb{R}^{N \times d} \\ h^{t+1} &= \tilde{h}^t + \text{MLP}(\text{LN}(\tilde{h}^t)) \in \mathbb{R}^{N \times d} \\ \text{with } \text{gMHA}(h, e) &= \text{Concat}_{i=1}^H (\text{gHA}(h_k, e_k)) W_O \in \mathbb{R}^{N \times d}, h_k \in \mathbb{R}^{N \times d-d/H}, e_k \in \mathbb{R}^{E \times d}, W_O \in \mathbb{R}^{d \times d} \\ \text{gHA}(h, e)_i &= \sum_{j \in N_i} \frac{\exp(q_i^T \text{diag}(e_{ij}) k_j / \sqrt{d})}{\sum_{j \in N_i} \exp(q_i^T \text{diag}(e_{ij}) k_j / \sqrt{d})} v_j \quad (\text{point-wise equation}) \\ &\quad \text{graph attention score w/ edge feature } e_{ij} \\ Q &= h_k W_Q, K = h_k W_K, V = h_k W_V \in \mathbb{R}^{N \times d-d/H}, E = e_k W_E \in \mathbb{R}^{E \times d-d/H}, W_Q, W_K, W_V, W_E \in \mathbb{R}^{d \times d} \\ h^{t=0} &= \text{LL}_1(h_0) + \text{LL}_2(p_0) \in \mathbb{R}^{N \times d} \quad (\text{input node feature and positional encoding}) \\ \text{with } p_0 &= \Phi_{(2, \dots, K+1)} \in \mathbb{R}^{N \times K}, \Delta = \Phi \Delta \Phi^T \in \mathbb{R}^{N \times N}\end{aligned}$$

Edge update equation:

$$\begin{aligned}e^t &= e^t + \text{gMHE}(\text{LN}(e^t), \text{LN}(h^t)) \in \mathbb{R}^{E \times d} \\ e^{t+1} &= e^t + \text{MLP}(\text{LN}(e^t)) \in \mathbb{R}^{E \times d} \\ \text{with } \text{gMHE}(e, h) &= \text{Concat}_{i=1}^H (\text{gHE}(e_k, h_k)) W_O' \in \mathbb{R}^{E \times d}, h_k \in \mathbb{R}^{N \times d-d/H}, e_k \in \mathbb{R}^{E \times d}, W_O' \in \mathbb{R}^{d \times d} \\ \text{with } \text{gHE}(e, h)_{ij} &= q_i \odot e_{ij} \odot k_j / \sqrt{d} \in \mathbb{R}^d \quad (\text{point-wise equation}) \\ e^{t=0} &= \text{LL}(e_0) \in \mathbb{R}^{E \times d} \quad (\text{input edge feature})\end{aligned}$$

```
# class graph multi head attention layer
class graph_MHA_layer(nn.Module): # MHA = Multi Head Attention
    def __init__(self, hidden_dim, head_hidden_dim, num_heads): # hidden_dim = d
        super().__init__()
        self.head_hidden_dim = head_hidden_dim # head_hidden_dim = d' = d/K
        self.num_heads = num_heads # number of heads = K
        self.W_Q = nn.Linear(hidden_dim, head_hidden_dim, bias=True) # define K x W matrix of size
        self.W_K = nn.Linear(hidden_dim, head_hidden_dim, bias=True)
        self.W_V = nn.Linear(hidden_dim, head_hidden_dim, bias=True)
        self.W_E = nn.Linear(hidden_dim, head_hidden_dim, bias=True)
```



Pros: very fast ∵ graph is small
Cons: memory limit (graph representation
is $O(n^2)$ but very sparse)

During implementation, we must learn an edge representation
for two nodes that are unconnected
Analogous to a newline "␣" token in NLP.
We use this edge feature for all edges $(v_i, v_j) \notin \text{Edges}$

(?) See lab code !!!

connect every node to every other node → No oversquashing problem

Lab 4 : Dense GT

No graph inductive bias

cannot do this with large graphs
... not enough datapoints to learn all params.

• GT with edge features and PyTorch

Lecture : Graph Transformers & Graph ViT

Lab 04 : Graph Transformers with edge features and PyTorch (dense linear algebra)

Xavier Bresson

Dwivedi, Bresson, A generalization of transformer networks to graphs, 2020
<https://arxiv.org/pdf/2012.09699.pdf>

```
# For Google Colaboratory
import sys, os
if 'google.colab' in sys.modules:
    # mount google drive
    from google.colab import drive
    drive.mount('/content/gdrive')
    path_to_file = '/content/gdrive/My Drive/CS5284_2024_codes/codes/10_Graph_Transformers'
    print(path_to_file)
    # change current path to the folder containing "path_to_file"
    os.chdir(path_to_file)
    !pwd
    !pip install dgl # Install DGL
```

```
# Libraries
import pickle
import matplotlib.pyplot as plt
import torch
from torch.utils.data import DataLoader
import torch.nn as nn
import time
import networkx as nx
import sys; sys.path.insert(0, 'lib/')
from lib.utils import compute_ncut
from lib.molecules import Dictionary, MoleculeDataset, MoleculeDGL, Molecule
import os, datetime
from rdkit import Chem
from rdkit.Chem import Descriptors
from rdkit.Chem import rdmolops
from rdkit import RDLogger
```

Design the class of dense GraphTransformer networks with edge features

Node update equation:

$$\begin{aligned}\tilde{h}^l &= h^l + \text{gMHA}(\text{LN}(h^l), \text{LN}(e^l)) \in \mathbb{R}^{N \times d} \\ h^{l+1} &= \tilde{h}^l + \text{MLP}(\text{LN}(\tilde{h}^l)) \in \mathbb{R}^{N \times d} \\ \text{with } \text{gMHA}(h, e) &= \text{Concat}_{k=1}^H (\text{gHA}(h_k, e_k)) W_O \in \mathbb{R}^{N \times d}, h_k \in \mathbb{R}^{N \times d' \times H}, e_k \in \mathbb{R}^{N \times N \times d'}, W_O \in \mathbb{R}^{d \times d} \\ \text{gHA}(h, e)_i &= \sum_{j \in V} \frac{\exp(q_i^T \text{diag}(e_{ij}) k_j / \sqrt{d'})}{\sum_{j \in V} \exp(q_i^T \text{diag}(e_{ij}) k_j / \sqrt{d'})} v_j \quad (\text{point-wise equation}) \\ &\quad \text{graph attention score w/ edge feature}_{ij} \\ Q &= h_k W_Q, K = h_k W_K, V = h_k W_V \in \mathbb{R}^{N \times d' \times H}, E = e_k W_E \in \mathbb{R}^{N \times N \times d'}, W_Q, W_K, W_V, W_E \in \mathbb{R}^{d' \times d} \\ h^{l=0} &= \text{LL}_1(h_0) + \text{LL}_2(p_0) \in \mathbb{R}^{N \times d} \quad (\text{input node feature and positional encoding}) \\ \text{with } p_0 &= \Phi_{\{2, \dots, K+1\}} \in \mathbb{R}^{N \times K}, \Delta = \Phi \Lambda \Phi^T \in \mathbb{R}^{N \times N}\end{aligned}$$

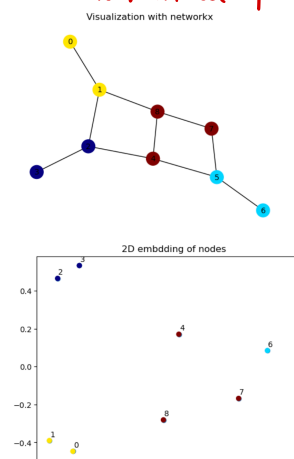
Edge update equation:

$$\begin{aligned}\tilde{e}^l &= e^l + \text{gMHE}(\text{LN}(e^l), \text{LN}(h^l)) \in \mathbb{R}^{N \times N \times d} \\ e^{l+1} &= \tilde{e}^l + \text{MLP}(\text{LN}(\tilde{e}^l)) \in \mathbb{R}^{N \times N \times d} \\ \text{with } \text{gMHE}(e, h) &= \text{Concat}_{k=1}^H (\text{gHE}(e_k, h_k)) W_O^e \in \mathbb{R}^{N \times N \times d}, h_k \in \mathbb{R}^{N \times d' \times H}, e_k \in \mathbb{R}^{N \times N \times d'}, W_O^e \in \mathbb{R}^{d \times d} \\ \text{with } \text{gHE}(e, h)_{ij} &= q_i \odot e_{ij} \odot k_j / \sqrt{d'} \in \mathbb{R}^d \quad (\text{point-wise equation}) \\ e^{l=0} &= \text{LL}(e_0) \in \mathbb{R}^{N \times N \times d} \quad (\text{input edge feature})\end{aligned}$$

Differences between sparse GT and dense GT:

- The edge features are now dense : Sparse $e \in \mathbb{R}^{E \times d}$ to $e \in \mathbb{R}^{N \times N \times d}$.
- The absence of edges in the graph is now considered as a bond type e.g. `None` type with integer value `0`.
- The attention function is now connected to all nodes in the graph : Sparse attention with $\sum_{j \in N_i}$ to dense attention $\sum_{j \in V}$.

```
# Global constants
num_heads = 8; d = 16 * num_heads; num_layers = 4; drop = 0.0; bs = 50
print('d, num_heads, num_layers, drop : ', d, num_heads, num_layers, drop)
```



All our ways to introduce inductive bias
ie. Encode graph structure

Different graph attention functions

- Graph-free attention^[1] : $\text{HA}(h) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d'}} \right) V \in \mathbb{R}^{N \times d'}$
- Standard graph attention^[2] : $\text{gHA}(h) = \text{Softmax} \left(A_G \odot \frac{QK^T}{\sqrt{d'}} \right) V \in \mathbb{R}^{N \times d'}$, $A_{Gij} = \begin{cases} A_{ij}^{\text{original}} & \text{if } i \sim j \\ 1 & \text{for } j = i \\ -\infty & \text{if } i \not\sim j \end{cases}$
- Kernel graph attention^[3] : $\text{gHA}(h) = \text{Softmax} \left(A_G \odot \frac{QK^T}{\sqrt{d'}} \right) V \in \mathbb{R}^{N \times d'}$, $A_{Gij} = A_{ij}^{\text{kernel}}$
- Additive graph attention^[4] : $\text{gHA}(h) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d'}} + D \right) V \in \mathbb{R}^{N \times d'}$, $D_{ij} = \text{learnable shortest path } (i, j)$
- Hadamard graph attention^[5] : $\text{gHA}(h) = A_G \odot \text{Softmax} \left(\frac{QK^T}{\sqrt{d'}} \right) V \in \mathbb{R}^{N \times d'}$, $A_{Gij} = A_{ij}^{\text{original}}$

Learns the representation
of the shortest path
somehow

SOTA, no one knows why

- [1] Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin, Attention is all you need, 2017
- [2] Dwivedi, Bresson, A generalization of transformer networks to graphs, 2020
- [3] Mialon, Chen, Selosse, Mairal, GraphiT: Encoding Graph Structure in Transformers, 2021
- [4] Ying, Cai, Luo, Zheng, Ke, He, Shen, Liu, Do Transformers Really Perform Bad for Graph Representation?, 2021
- [5] He, Hooi, Laurent, Perold, LeCun, Bresson, A Generalization of ViT/MLP-Mixer to Graphs, 2022

Graph Transformers

- Material

- GitHub repo (600*+) :

<https://github.com/graphdeeplearning/graphtransformer>

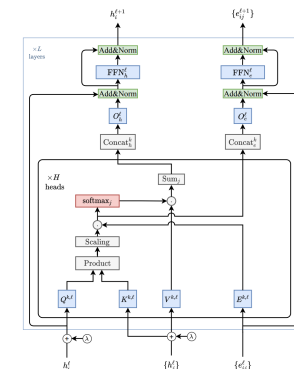
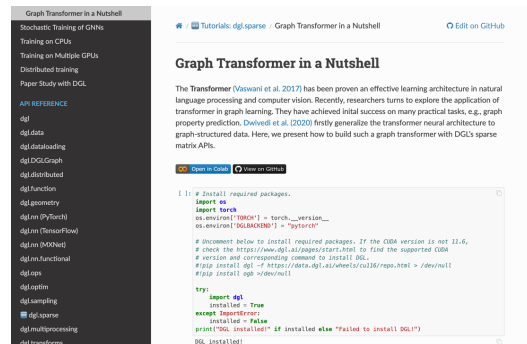
- Blogpost (Vijay Dwivedi) : <https://towardsdatascience.com/graph-transformer-generalization-of-transformers-to-graphs-ead2448cff8b>

- Andrew Ng's article : <https://www.deeplearning.ai/the-batch/issue-151>

DeepLearning.AI
THE BATCH

- DGL tutorial :

https://docs.dgl.ai/en/latest/notebooks/sparse/graph_transformer.html



Hybrid models

- Graph Transformer^[1] is a type of MP-GNN s.a. GCN^[2], GAT^[3], etc.
 - In practice, GT compares favorably to GCN, GAT on the benchmark^[4].
- However, all MP-GNNs are limited by **over-squashing** and **poor long-range interactions**.
 - As the number of layers increases, the local reception field grows exponentially, and aggregation functions struggle to distinguish critical from irrelevant information.★
- A well-known solution from NLP is to use (self-)attention to all nodes simultaneously.
 - This idea was a breakthrough in NLP for standard/fully-connected Transformers^[5] (T).
 - However, standard T do not perform well experimentally for graph learning tasks and have a higher complexity $O(N^2)$.
 - It is actually critical to use the graph topology as it offers a strong inductive bias for generalization.

∴ we now have

① Attention mechanism

② Positional encodings

③ Edge features

[1] Dwivedi, Bresson, A generalization of transformer networks to graphs, AAAI 2021

[2] Kipf, Welling, Semi-supervised classification with graph convolutional networks, 2017

[3] Velickovic, Cucurull, Casanova, Romero, Lio, Bengio, Graph Attention Networks, 2018

[4] Dwivedi, Joshi, Laurent, Bengio, Bresson, Benchmarking graph neural networks, 2020

[5] Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin, Attention is all you need, 2017

Hybrid models

oversquashing problem

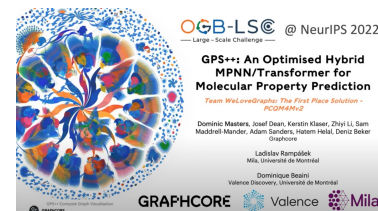
Has long-range message-passing

- This led to design hybrid models **mixing GT+T**, potentially solving over-squashing/long-range issues while retaining the inductive bias of graph topology.
- Hybrid GT+T architectures have been showed successful in practice.
 - Winner^[1] and runner-up^[2] at the NeurIPS'22 OGB Large-Scale Challenge competition^[3]

OGB-LS[©] @ NeurIPS 2022
— Large - Scale Challenge —

- 10:50PM–11:40PM, UTC: Graph-Level Track (PCQM4Mv2)

- Intro to the task (5min) [\[Video\]](#)
- Live presentation by 1st place winner: **WeLoveGraphs** (10min) [\[Video\]](#)
- Live presentation by 2nd place winner: **NVIDIA-PCQM4Mv2** (10min) [\[Video\]](#)
- Live presentation by 2nd place winner: **VISNet** (10min) [\[Video\]](#)
- Live Q&A and discussion (15min) [\[Video\]](#)



- [1] Masters et.al, GPS++: An Optimised Hybrid MPNN/Transformer for Molecular Property Prediction, 2022
 [2] Darabi et-al, Heterogenous Ensemble of Models for Molecular Property Prediction, 2022
 [3] <https://ogb.stanford.edu/neurips2022/workshop>

*cons: very unstable
very irregular/non smooth
learned manifold*

Hybrid models

- Hybrid GT+T models can be appealing but
 - Their complexity is higher $O(N^2+E)$ ^[1,2]. Linear approximation of the attention mechanism^[3] can be used to reduce the complexity of GT+T to $O(N+E)$ ^[4].
 - Hybrid architectures are more complex models, often less stable to train and require more hyper-parameter tuning (i.e. industry resources). A similar example is LSTM+T^[5] in NLP, which was SOTA until 2016 and LSTM was later discarded for more stable/scalable deep T.
 - Hybrid architectures can be any MP-GNN+T combination.
- In my opinion
 - Hybridization MP-GNN+T is temporary and new models, simpler and more stable, will be developed, similar to what has happened in NLP. See next sections.
- Survey^[6] on the development of GT-based models
 - Study properties of isomorphism expressivity, graph topology as good inductive bias, reduce over-squashing and attention interpretability in GT architectures.

[1] Kreuzer, Beaini, Hamilton, Letourneau, Tossou, Rethinking Graph Transformers with Spectral Attention, 2021

[2] Dwivedi, Luu, Laurent, Bengio, Bresson, Graph neural networks with learnable structural and positional representations, 2021

[3] Choromanski et-al, Rethinking attention with performers, 2021

[4] Rampasek et-al, Recipe for a General, Powerful, Scalable Graph Transformer, 2022

[5] Wu et-al, Google's neural machine translation system: Bridging the gap between human and machine translation, 2016

[6] Muller, Galkin, Morris, Rampasek, Attending to Graph Transformers, 2023

Outline

- Review of graph network architectures
- Graph positional encoding
- Transformers for sequences
- Graph Transformers
- **ViT/MLP-Mixer for images**
- Graph ViT/MLP-Mixer
- Conclusion

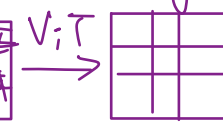
Transformers for long sequences

- Transformers^[1] have been a breakthrough in NLP but the attention mechanism is computationally expansive since the memory/speed complexity is $O(N^2d)$.
- This limits Transformer on GPUs to short sequences, $N \leq 1,000$ for $d=512$.
- How to scale-up to long sequences?

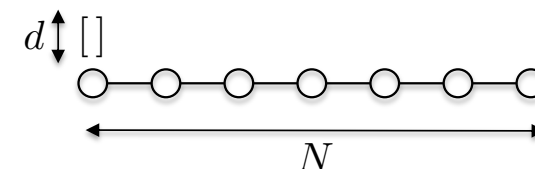
- Linear approximations of the attention function^[2,3].

$$\text{Attn}(Q, K, V) := \text{Softmax} \left[\frac{QK^T}{\sqrt{d}} \right] V \leftarrow O(n^2)$$

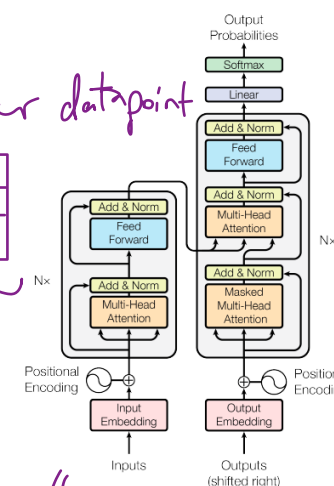
Linear approximation techniques like "low-rank matrix multiplication"



Do attention on these patches instead



$$h \in \mathbb{R}^{N \times d}$$



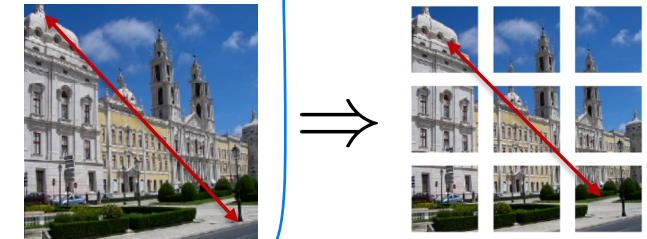
- [1] Vaswani, Shazeer, Parmar, Uszkoreit, Jones, Gomez, Kaiser, Polosukhin, Attention is all you need, 2017
- [2] Choromanski et-al, Rethinking attention with performers, 2020
- [3] Jaegle, Gimeno, Brock, Vinyals, Zisserman, Carreira, Perceiver: General perception with iterative attention, 2021

use it for doing downstream tasks (e.g. image classification, regression, similarity matching, etc.)

After updating whole image embedding with L transformer layers that perform message passing with each patch, the CLS token will be a good summarization of the whole image

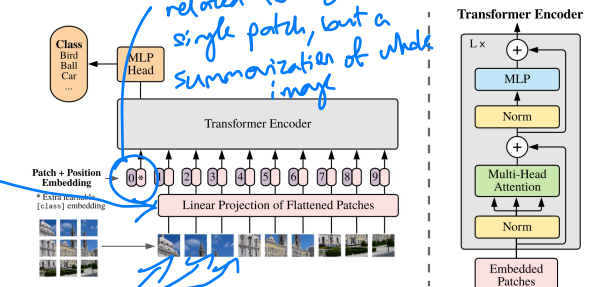
ViT/MLP-Mixer for images

- How to scale Transformer to image grids (CV)?
E.g. $N = 1,000 \times 1,000$ pixels = 1M pixels/nodes
- ViT architecture^[1] :
 - Perform attention not on image pixels but on image patches : $O(N^2d) \searrow O(P^2d)$ w/ $P \ll N$ patches
 - Embed patches with MLP
 - Same layer normalization + residual connection + MLP
 - Raster ordering of patches as node positional encoding

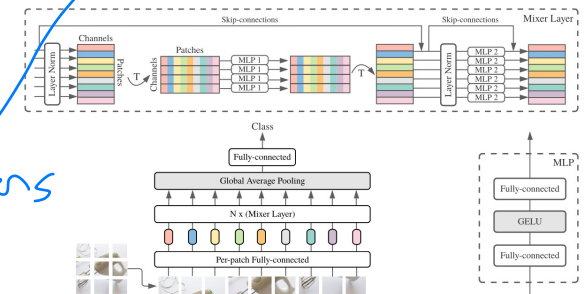


N pixels P << N patches

CLS token: Not related to any single patch, but a summarization of whole image



Vision Transformer (ViT)



MLP-Mixer

visual tokens

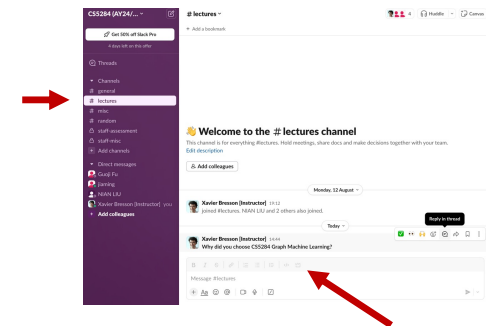
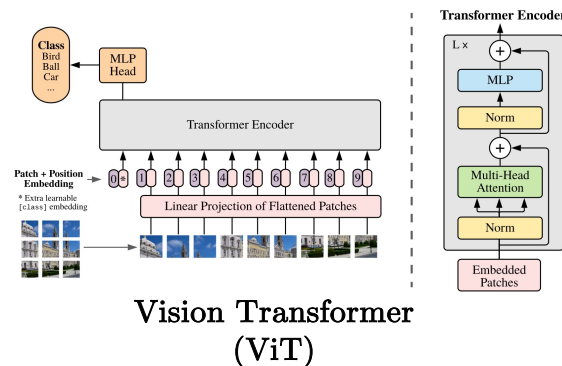
[1] Dosovitskiy et-al, An image is worth 16x16 words: Transformers for image recognition at scale, 2020
[2] Tolstikhin et-al, MLP-mixer: An all-MLP architecture for vision, 2021

In-lecture question

- ViTs are primarily based on Transformer layers, with the initial layer using an MLP to embed image patches. Does this mean ViTs lack convolutional layers?
- What is currently the most efficient vision architecture in practice -- ViTs or CNNs?
- In Slack #lectures
 - Identify the question and Reply in thread with a short response
- Answer :

Hybrid:
many initial CNN
layers, then later
use transformers to
find the most relevant
features extracted

much faster

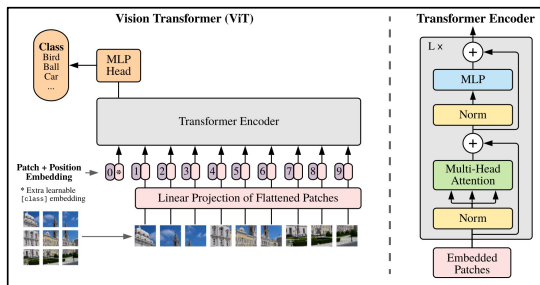


Outline

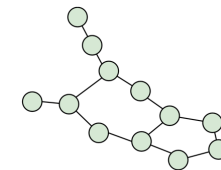
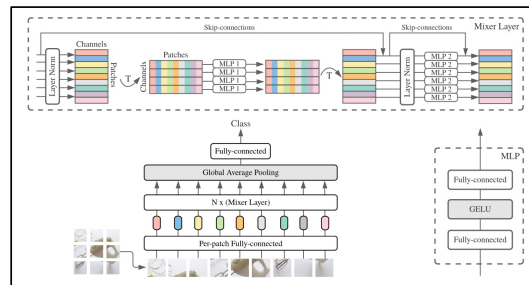
- Review of graph network architectures
- Graph positional encoding
- Transformers for sequences
- Graph Transformers
- ViT/MLP-Mixer for images
- **Graph ViT/MLP-Mixer**
- Conclusion

Why ViT/MLP-Mixer for graphs

- Standard ViT/MLP-Mixer^[2] captures long-range interaction in images with low complexity.
- Our goal is to transfer these advantages to GNNs and design a new network that simultaneously
 - captures long-range dependency (mitigating the over-squashing issue),
 - keeps linear speed/memory complexity (similar to standard MP-GNNs),
 - achieves isomorphism expressivity (good representation power).
- However, generalizing ViT/MLP-Mixer is challenging due to the variable nature of graphs.



ViT/MLP-Mixer for images^[1,2]



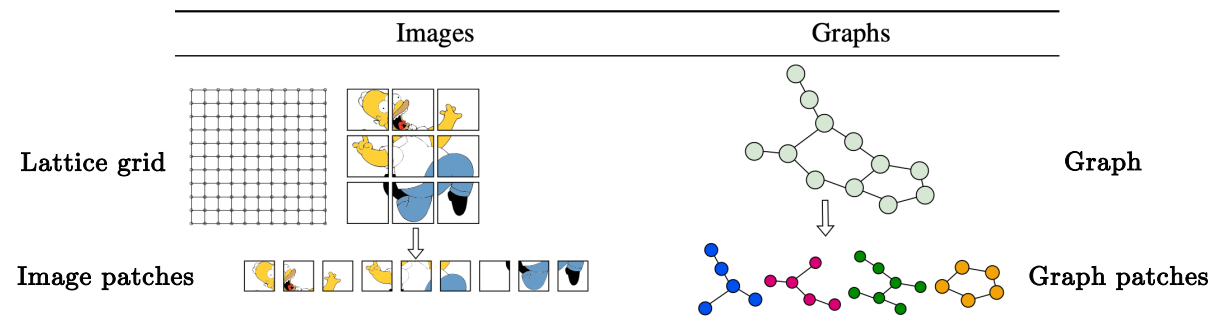
ViT/MLP-Mixer for graphs

[1] Dosovitskiy et-al, An image is worth 16x16 words: Transformers for image recognition at scale, 2020

[2] Tolstikhin et-al, MLP-mixer: An all-MLP architecture for vision, 2021

Generalization challenges

- We identify four key challenges to generalize ViT/MLP-Mixer from grids to graphs.
 - (1) How to define and quickly extract graph patches(/tokens)?
 - Images are supported by a regular lattice, easily split into same-size grid-like patches via fast pixel reordering.
 - Graphs are irregular and cannot be divided into similar patches. *solution: Metis*
 - (2) How to encode graph patches into a vectorial representation?
 - Same-size patches can be encoded with MLP. *solution: GNN*
 - Graph patches have different topological structures (variable #nodes and #edges)



Generalization challenges

(3) How to preserve node and patch positional information?

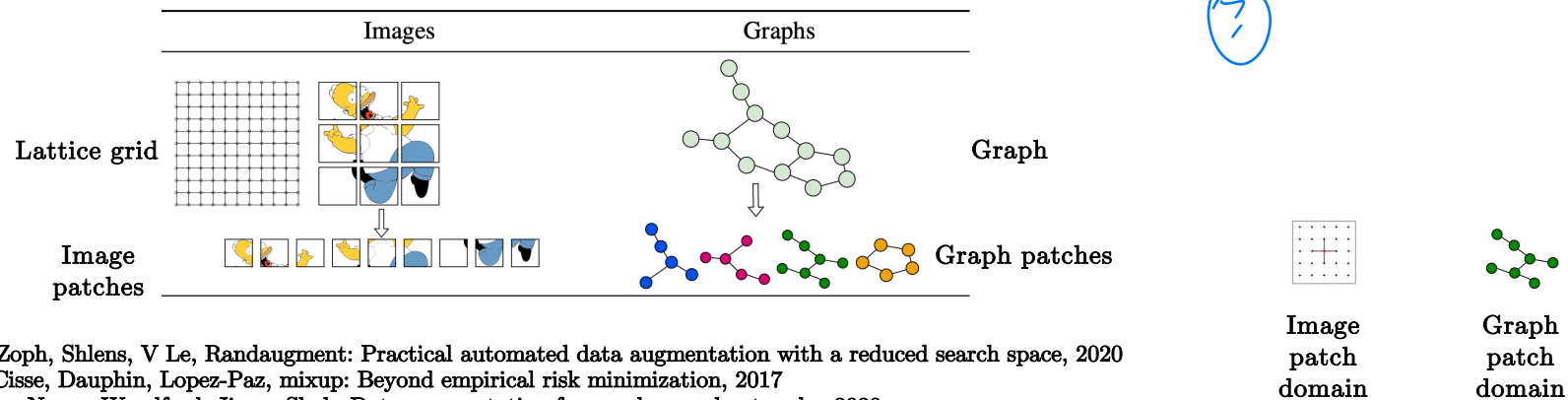
solution: Average of Laplacian PE for all nodes in a partition

- Image patches have implicit positions since images are always ordered the same way, but graphs are naturally not-aligned and the set of graph patches are therefore unordered.
- Pixels in each patch are ordered the same way, but nodes in graph tokens are generally unordered.

(4) How to reduce over-fitting for graphs?

- ViT/MLP-Mixer architectures are strong over-fitters. Images have a rich set of data augmentation and regularization techniques, e.g. cropping, flipping, RandAugment^[1], mixup^[2].
- Graph data augmentation methods^[3] are not yet as effective. *solution:*

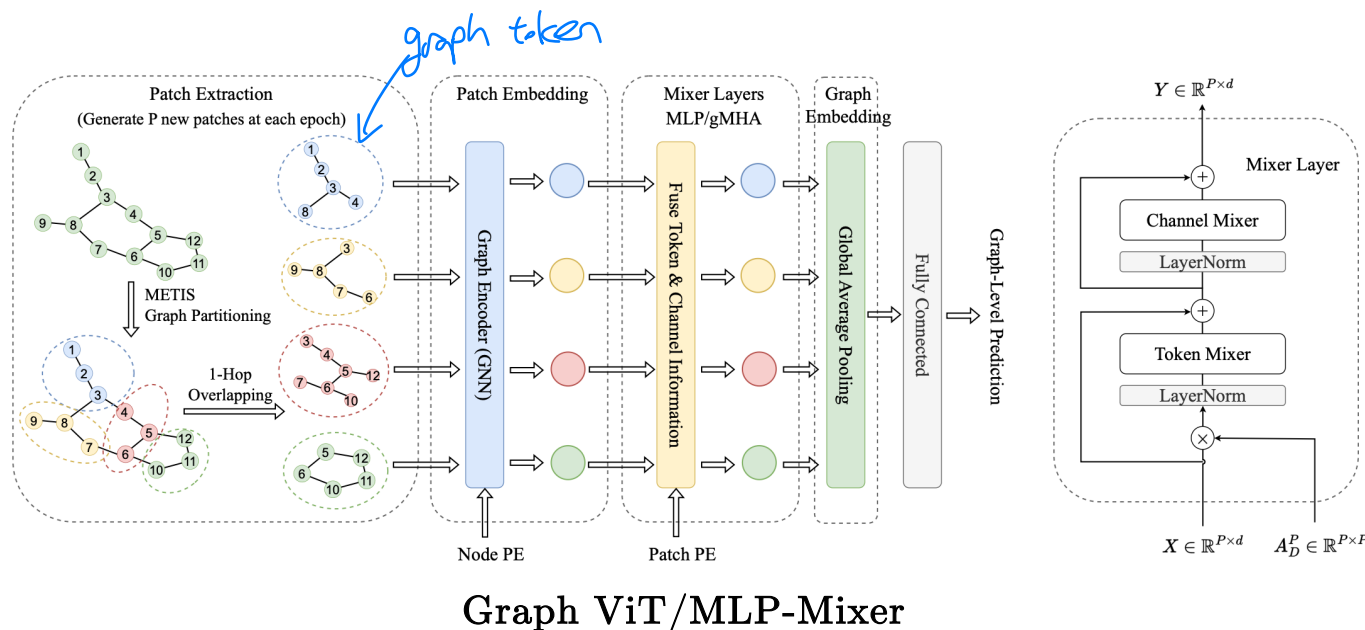
(?)



- [1] Cubuk, Zoph, Shlens, V Le, RandAugment: Practical automated data augmentation with a reduced search space, 2020
 [2] Zhang, Cisse, Dauphin, Lopez-Paz, mixup: Beyond empirical risk minimization, 2017
 [3] Zhao, Liu, Neves, Woodford, Jiang, Shah, Data augmentation for graph neural networks, 2020

Proposed architecture

- Generic architecture of Graph ViT/MLP-Mixer^[1] is illustrated below.
- Implementation of each building block is arbitrary.
- Our choices lead to a simple framework that provides speed and accuracy.

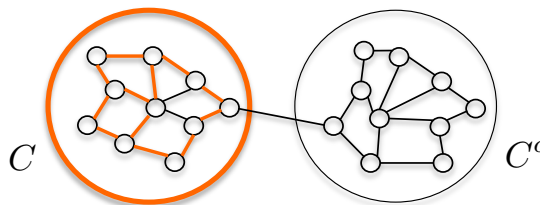


[1] He, Hooi, Laurent, Perold, LeCun, Bresson, A Generalization of ViT/MLP-Mixer to Graphs, 2022

skipped

Patch extraction

- Graph patch extraction algorithm must satisfy the following conditions :
 - The clustering algorithm can be applied to any arbitrary graph.
 - The nodes in the patch must be more connected than for those outside the patch (s.a. compound of atoms).
 - The extraction must be fast, i.e. complexity at most linear w.r.t. the number of edges $O(E)$.
- Graph partitioning algorithms have a long and rich development^[1,2].
- They are NP-hard combinatorial problems and approximations are required.
- We select Metis^[3] that
 - Partitions a graph by maximizing the number of within-cluster links.
 - Provides one of the best trade-off accuracy and speed.



$$\max_C \frac{\text{Assoc}(C, C)}{\text{Vol}(C)} + \frac{\text{Assoc}(C^c, C^c)}{\text{Vol}(C^c)}$$
$$\text{Assoc}(C, C) = \sum_{i \in C, j \in C} A_{ij} \quad (\text{Adj. matrix})$$
$$\text{Vol}(C) = \sum_{i \in C} d_i \quad (\text{degree})$$

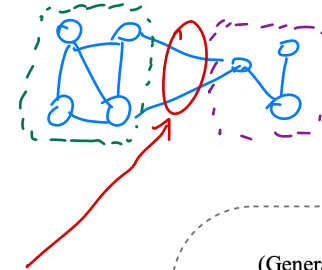
Normalized Association

[1] Von Luxburg, A tutorial on spectral clustering, 2007

[2] Buluc, Meyerhenke, Safro, Sanders, Schulz, Recent advances in graph partitioning, 2016

[3] Karypis, Kumar, A fast and high quality multilevel scheme for partitioning irregular graphs, 1998

Metis limitations



- Two major limitations

(1) Metis produces non-overlapping patches and **loses the cutting edges**. We expand the graph patches with a k-hop neighborhood.

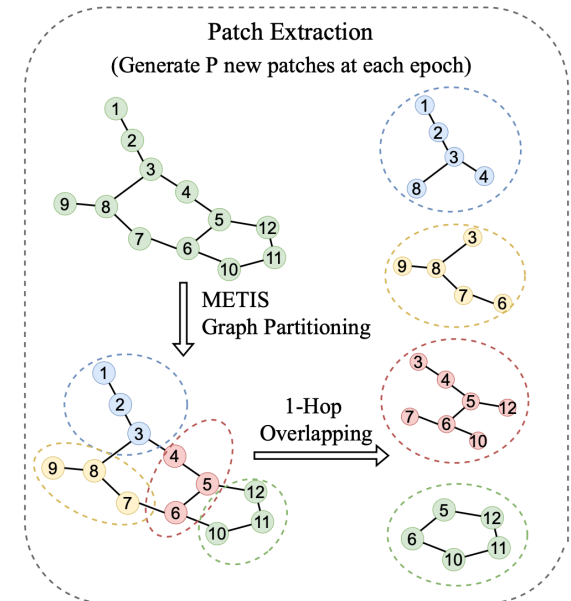
(2) Metis is a deterministic process, it always generates the same clusters which leads to **over-fitting**. why?

- Data augmentation

- At each epoch, Metis is applied on a perturbed graph (by randomly dropping a small set of edges) to get slightly different partitions.

- Then **graph patches are extracted from the original graph** (not the perturbed one) to retain the original nodes and edges.

- This new dropout technique produces distinct graph patches at each epoch that significantly improves the results at a small additional computational time.



Patch encoder

- Image patch embedding is fast and well-defined with a simple MLP.
- Graph patch embedding must handle heterogeneous structure, variable patch size, index permutation invariance, weight sharing and capable of distinguishing isomorphic patches.
- As a result, the graph patch encoder is a **MP-GNN (e.g. GCN^[1], GAT^[2], GT^[3])** which can map a graph token into a fixed-size representation into 3 steps :

(1) Raw node and edge embedding :

$$h_i^0 = T^0 p_i + U^0 \alpha_i + u^0 \in \mathbb{R}^d, \quad e_{ij}^0 = V^0 \beta_{ij} + v^0 \in \mathbb{R}^d$$

Positional encoding
Raw node features
Raw edge features

(2) Graph convolutional layers with MP-GNN :

$$h_{i,p}^{\ell+1} = f_{\text{node}}(h_{i,p}^{\ell}, \{h_{j,p}^{\ell}, e_{ij,p}^{\ell} | j \in \mathcal{N}(i)\}) + g_{\text{patch-n}}(h_p^{\ell}) \in \mathbb{R}^d$$

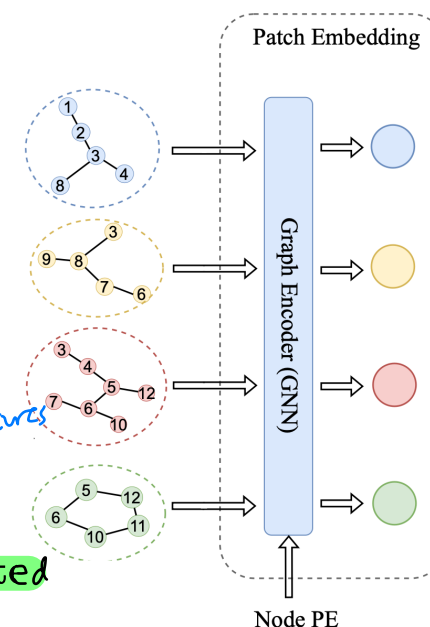
$$e_{ij,p}^{\ell+1} = f_{\text{edge}}(h_{i,p}^{\ell}, h_{j,p}^{\ell}, e_{ij,p}^{\ell}) + g_{\text{patch-e}}(e_p^{\ell}) \in \mathbb{R}^d$$

(3) Pooling and readout : $x_{G_p} = \text{MLP}(h_p) \in \mathbb{R}^d$,

$$h_p = \frac{1}{|\mathcal{V}_p|} \sum_{i \in \mathcal{V}_p} h_{i,p}^{\ell=L} \in \mathbb{R}^d$$

Patch embedding
mean of node features

- Note that the MP-GNN is applied to small patch graphs, which are **not affected by poor long-range interaction and over-squashing**.



[1] Kipf, Welling, Semi-supervised classification with graph convolutional networks, 2017
 [2] Velickovic, Cucurull, Casanova, Romero, Lio, Bengio, Graph Attention Networks, 2018
 [3] Dwivedi, Bresson, A generalization of transformer networks to graphs, 2021

Positional information

- Regular grid offers an implicit arrangement for the sequence of image patches and for the pixels inside the image patch.
- General graphs do not have a canonical ordering of nodes and patches. This lack of positional information reduces the expressivity of the network.
- We use two explicit positional encoding (PE) :
 - Node PE : We use random-walk structural encoding^[1] for molecular data and Laplacian eigenvectors^[2,3] encodings for synthetic graph datasets.
 - Patch PE : We use random-walk structural encoding^[1] on the the adjacency matrix of the patch graphs (computed from the original graph adjacency matrix and the cluster extracted by Metis) :

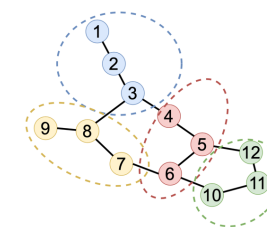
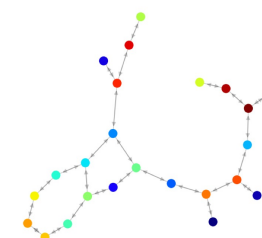
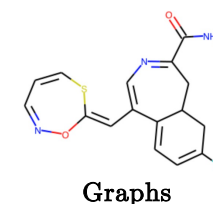
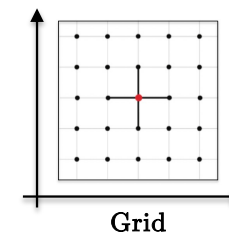
$$A_{\mathcal{D}}^P = (D^{-1} A^P)^n \in \mathbb{R}^{P \times P}$$

$$A_{ij}^P = |\mathcal{V}_i \cap \mathcal{V}_j| = \text{Cut}(\mathcal{V}_i, \mathcal{V}_j) = \sum_{k \in \mathcal{V}_i} \sum_{l \in \mathcal{V}_j} A_{kl}$$

[1] Dwivedi, Luu, Laurent, Bengio, Bresson, Graph neural networks with learnable structural and positional representations, 2021

[2] Dwivedi, Joshi, Laurent, Bengio, Bresson, Benchmarking graph neural networks, 2020

[3] Dwivedi, Bresson, A generalization of transformer networks to graphs, 2021



Mixer layer and graph embedding

- The original mixer layer^[1,2] is a network that alternates channel and token mixing steps with self-attention or MLP. These interleaved steps fuse token and channel information.
 - Self-attention mixer has complexity $O(P^2d)$ and MLP mixer $O(Pd)$.
 - However, P value is small, almost same speed in practice.
- Let $X \in \mathbb{R}^{P \times d}$ be the patch embedding matrix, then the self-attention mixer layer is defined as

$$U = X + \text{gMHA}(\text{LayerNorm}(X)) \in \mathbb{R}^{P \times d}$$

Token mixer

$$Y = U + \text{MLP}(\text{LayerNorm}(U)) \in \mathbb{R}^{P \times d}$$

Channel mixer

and the MLP mixer layer is defined as

$$U = X + (W_2 \sigma(W_1 \text{LayerNorm}(X))) \in \mathbb{R}^{P \times d}$$

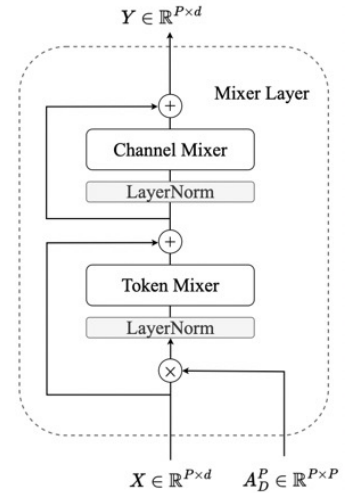
Token mixer

$$Y = U + (W_4 \sigma(W_3 \text{LayerNorm}(U)^T))^T \in \mathbb{R}^{P \times d}$$

Channel mixer

- The final graph-level representation is given by mean pooling all the (non-empty) patches and using a small MLP :

$$y_G = \text{MLP}(h_G) \in \mathbb{R}/\mathbb{R}^{n_c}, \quad h_G = \sum_p m_p \cdot x_{G_p} / \sum_p m_p \in \mathbb{R}^d$$



[1] Dosovitskiy et-al, An image is worth 16x16 words: Transformers for image recognition at scale, 2020

[2] Tolstikhin et-al, MLP-mixer: An all-MLP architecture for vision, 2021

Numerical experiments

- We evaluate Graph ViT/MLP-Mixer on a range of benchmark graph datasets :
 - 1) Simulated datasets : CSL^[1], EXP^[2], SR25^[3] and TreeNeighbourMatch^[4] datasets.
 - 2) Small molecular datasets : ZINC from Benchmarking GNNs^[5] (for solubility) and MolTOX21 (for toxicity) and MolHIV from Open Graph Benchmark (OGB)^[6] (for HIV inhibition).
 - 3) Large molecular datasets : Peptides-func (for antibacterial, antiviral properties etc) and Peptides-struct (for 3D properties) from Long Range Graph Benchmark (LRGB)^[7]

[1] Murphy, Srinivasan, Rao, Ribeiro, Relational pooling for graph representations, 2019

[2] Abboud, Ceylan, Grohe, Lukaszewicz, The surprising power of graph neural networks with random node initialization, 2020

[3] Balcilar, Heroux, Gauzere, Vasseur, Adam, Honeine, Breaking the limits of message passing graph neural networks, 2021

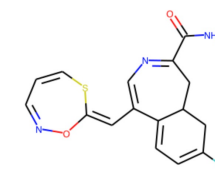
[4] Alon, Yahav, On the bottleneck of graph neural networks and its practical implications, 2020

[5] Dwivedi, Joshi, Laurent, Bengio, Bresson, Benchmarking graph neural networks, 2020

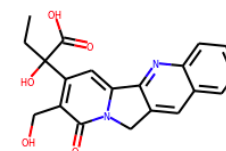
[6] Hu et-al, Open graph benchmark: Datasets for machine learning on graphs, 2020

[7] Dwivedi, Rampavek, Galkin, Parviz, Wolf, Luu, Beaini, Long range graph benchmark, 2022

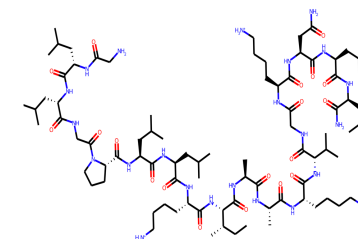
Benchmark datasets



ZINC



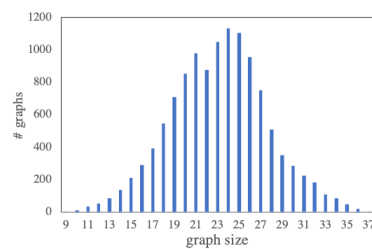
MolHIV



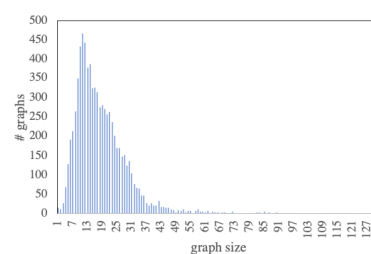
Peptide

Statistics of molecular and synthetic datasets :

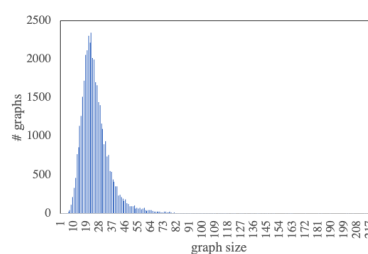
	Dataset	#Graphs	#Nodes	Avg. #Nodes	Avg. #Edges	Task	Metric
Simulated datasets	CSL	150	41	41	164	10-class classif.	Accuracy
	EXP	1,200	32-64	44.4	110.2	2-class classif.	Accuracy
	SR25	15	25	25	300	15-class classif.	Accuracy
	TreeNeighbourMatch (r=2)	96	7	7	6	4-class classif.	Accuracy
	TreeNeighbourMatch (r=3)	32,000	15	15	14	8-class classif.	Accuracy
	TreeNeighbourMatch (r=4)	64,000	31	31	30	16-class classif.	Accuracy
	TreeNeighbourMatch (r=5)	128,000	63	63	62	32-class classif.	Accuracy
	TreeNeighbourMatch (r=6)	256,000	127	127	126	64-class classif.	Accuracy
	TreeNeighbourMatch (r=7)	512,000	255	255	254	128-class classif.	Accuracy
Small molecular datasets	TreeNeighbourMatch (r=8)	640,000	511	511	510	256-class classif.	Accuracy
	ZINC	12,000	9-37	23.2	24.9	regression	MAE
	MolTOX21	7,831	1-132	18.57	38.6	12-task classif.	ROCAUC
Large molecular datasets	MolHIV	41,127	2-222	25.5	54.9	binary classif.	ROCAUC
	Peptides-func	15,535	8-444	150.9	307.3	10-class classif.	Avg. Precision
	Peptides-struct	15,535	8-444	150.9	307.3	regression	MAE



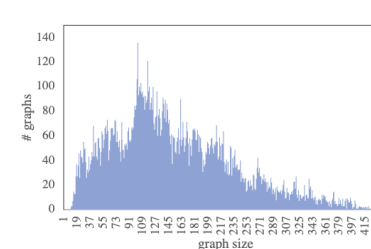
ZINC



MolTOX21



MolHIV



Peptides-func/struct

Comparison with standard MP-GNNs

- Graph ViT/MLP-Mixer lifts the performance of all base MP-GNNs across all datasets, which include GCN^[1], GatedGCN^[2], GINE^[3] and GraphTransformer^[4].
- We augmented all base models with the same PEs as Graph ViT/MLP-Mixer to ensure fair comparison.
- These promising results demonstrate the generic nature of our architecture which can be applied to any MP-GNN in practice.

Model	ZINC	MolTOX21	MolHIV	Peptide-func	Peptide-struct
	MAE ↓	ROCAUC ↑	ROCAUC ↑	AP ↑	MAE ↓
GCN	0.1952 ± 0.0057	0.7525 ± 0.0031	0.7813 ± 0.0081	0.6328 ± 0.0086	0.2758 ± 0.0012
GCN-MLP-Mixer	0.1347 ± 0.0020	0.7816 ± 0.0075	0.7929 ± 0.0111	0.6832 ± 0.0061	0.2486 ± 0.0041
GCN-ViT	0.1688 ± 0.0095	0.7820 ± 0.0096	0.7780 ± 0.0120	0.6855 ± 0.0049	0.2468 ± 0.0015
GatedGCN	0.1577 ± 0.0046	0.7641 ± 0.0057	0.7874 ± 0.0119	0.6300 ± 0.0029	0.2778 ± 0.0017
GatedGCN-MLP-Mixer	0.1244 ± 0.0053	0.7910 ± 0.0040	0.7976 ± 0.0136	0.6932 ± 0.0017	0.2508 ± 0.0007
GatedGCN-ViT	0.1421 ± 0.0031	0.7857 ± 0.0028	0.7734 ± 0.0114	0.6942 ± 0.0075	0.2465 ± 0.0015
GINE	0.1072 ± 0.0037	0.7730 ± 0.0064	0.7885 ± 0.0034	0.6405 ± 0.0077	0.2780 ± 0.0021
GINE-MLP-Mixer	0.0733 ± 0.0014	0.7868 ± 0.0043	0.7997 ± 0.0102	0.6970 ± 0.0080	0.2494 ± 0.0007
GINE-ViT	0.0849 ± 0.0047	0.7851 ± 0.0077	0.7792 ± 0.0149	0.6919 ± 0.0085	0.2449 ± 0.0016
GraphTrans	0.1230 ± 0.0018	0.7646 ± 0.0055	0.7884 ± 0.0104	0.6313 ± 0.0039	0.2777 ± 0.0025
GraphTrans-MLP-Mixer	0.0773 ± 0.0030	0.7817 ± 0.0040	0.7969 ± 0.0061	0.6858 ± 0.0062	0.2480 ± 0.0013
GraphTrans-ViT	0.0960 ± 0.0073	0.7835 ± 0.0032	0.7755 ± 0.0208	0.6876 ± 0.0059	0.2455 ± 0.0027

Table: Results are averaged over 4 runs with 4 different seeds.

[1] Kipf, Welling, Semi-supervised classification with graph convolutional networks, 2017

[2] Bresson, Laurent, Residual gated graph convnets, 2017

[3] Hu, Liu, Gomes, Zitnik, Liang, Pande, Leskovec, Strategies for pre-training graph neural networks, 2019

[4] Dwivedi, Bresson, A generalization of transformer networks to graphs, 2021

Comparison with SOTA

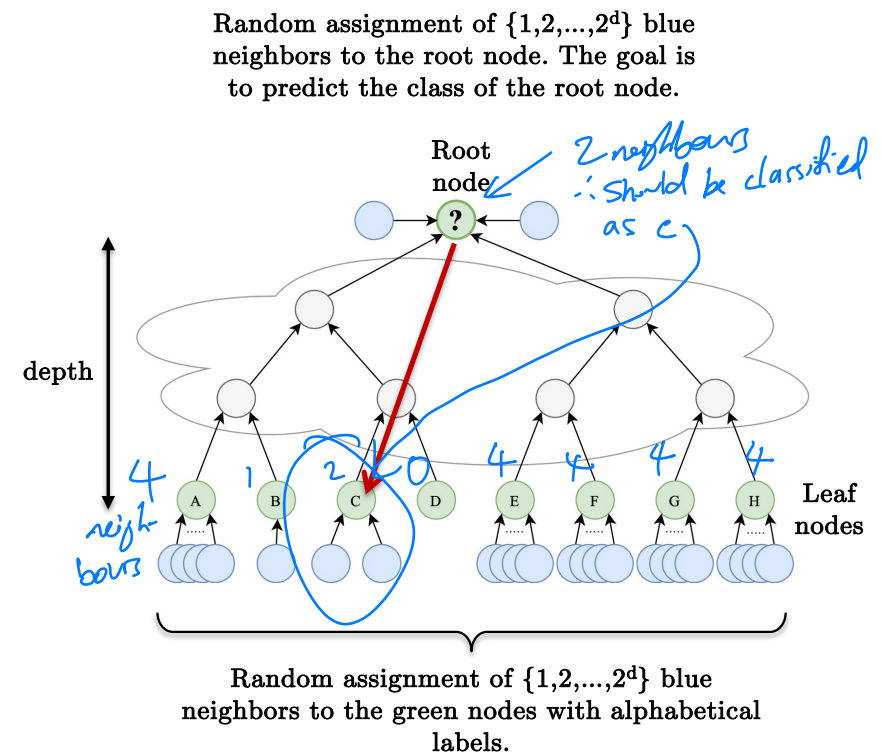
- We compare against GNN models with SOTA results
 - Graph Transformers: GraphiT, GPS, SAN, etc
 - Expressive GNNs: GNN-AK₊ and SUN
- For small molecular graphs, our model achieved competitive results on ZINC and MolHIV.
- For larger molecular graphs, our model sets SOTA performance on Peptides-fun/struct.
- Graph ViT/MLP-Mixer offers better space-time complexity and scalability.
 - SAN+LapPE and SUN require 8× and 43× training time per epoch, and 12× and 18× memory respectively, compared to our model.

Model	ZINC	MolHIV	Peptides-func			Peptides-struct		
	MAE ↓	ROCAUC ↑	AP ↑	Time	Mem.	MAE ↓	Time	Mem.
GT (Dwivedi et al., 2020)	0.226 ± 0.014	–	–	–	–	–	–	–
GraphiT (Mialon et al., 2021)	0.202 ± 0.011	–	–	–	–	–	–	–
Graphormer (Ying et al., 2021)	0.122 ± 0.006	–	–	–	–	–	–	–
GPS (Rampášek et al., 2022)	0.070 ± 0.004	0.7880 ± 0.0101	0.6562 ± 0.0115	1.4×	6.8×	0.2515 ± 0.0012	1.3×	8.3×
SAN+LapPE (Kreuzer et al., 2021)	0.139 ± 0.006	0.7775 ± 0.0061	0.6384 ± 0.0121	9.4×	12.4×	0.2683 ± 0.0043	8.8×	14.7×
SAN+RWSE (Kreuzer et al., 2021)	–	–	0.6439 ± 0.0075	8.0×	19.5×	0.2545 ± 0.0012	7.9×	14.5×
GNN-AK+ (Zhao et al., 2021)	0.080 ± 0.001	0.7961 ± 0.0119	0.6480 ± 0.0089	2.6×	7.8×	0.2736 ± 0.0007	2.5×	9.2×
SUN (Frasca et al., 2022)	0.084 ± 0.002	0.8003 ± 0.0055 ¹	0.6730 ± 0.0078	43.8×	18.8×	0.2498 ± 0.0008	42.7×	20.7×
CIN (Bodnar et al., 2021)	0.079 ± 0.006 ²	0.8094 ± 0.0057	–	–	–	–	–	–
Graph MLP-Mixer	0.073 ± 0.001	0.7997 ± 0.0102	0.6970 ± 0.0080	1.0×	1.0×	0.2475 ± 0.0015	1.0×	1.2×
Graph ViT	0.085 ± 0.005	0.7792 ± 0.0149	0.6942 ± 0.0075	1.1×	0.8×	0.2449 ± 0.0016	1.0×	1.0×

Table: Results are averaged over 4 runs with 4 different seeds.

Synthetic dataset for over-squashing

- TreeNeighboursMatch^[1] is a synthetic dataset to simulate the exponentially-growing receptive field in GNNs and controls the intensity of over-squashing with the tree depth.
- The experiment consists in classifying the root node with the letter of the green labeled node with the same number of blue neighbors than the root node.
- To succeed, the network must be able to communicate across long distance (from the root to the leaf) and extract the information from $O(2^{L+1})$ aggregated nodes.



[1] Alon, Yahav, On the bottleneck of graph neural networks and its practical implications, 2020

Graph ViT/MLP-Mixer mitigates over-squashing

- Standard MP-GNNs s.a. GCN, GGCN, GAT, GIN fail to generalize for depth > 4 because of over-squashing that squeezes too much info from the exponential growth of the tree reception field.
- Our model is able to mitigate over-squashing and generalize until depth = 7 since it transmits the long-distance information directly with the mixer layer.

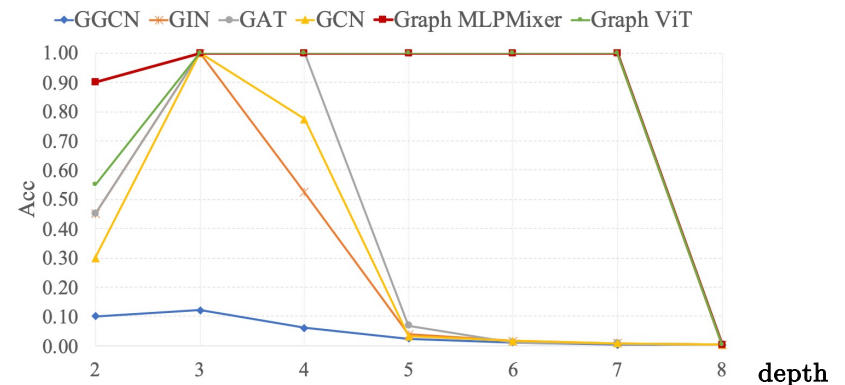
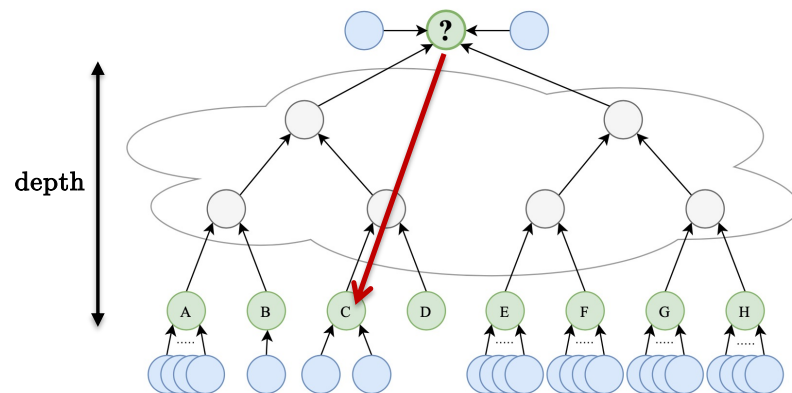
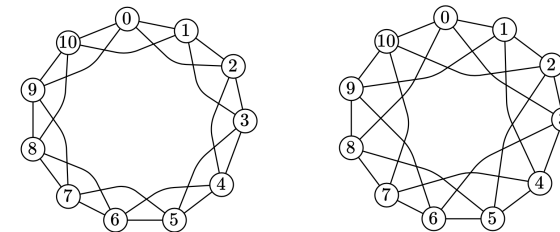


Figure. Test Accuracy w.r.t. the tree depth in the NEIGHBORSMATCH problem.

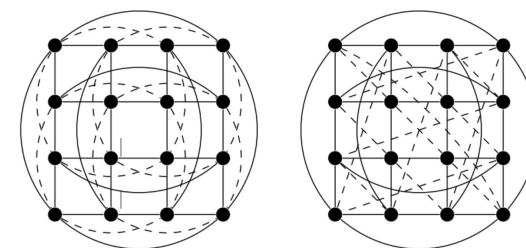
[1] Alon, Yahav, On the bottleneck of graph neural networks and its practical implications, 2020

Synthetic datasets for expressivity

- Circular Skip Link (CSL) dataset^[1]
 - Classify isomorphic graphs that differ by the skip link value.
 - CSL contains 150 4-regular graphs (1-WL failed) divided into 10 isomorphism classes.
- EXP dataset^[2]
 - EXP contains 1200 1&2-WL failed graphs that are split into two classes.
- SR25 dataset^[3]
 - SR25 has 15 strongly regular 3-WL failed graphs with 25 nodes each with the goal of classifying them.



Example of non-isomorphic graphs where 1-WL test failed to distinguish (w/ skip-link 2 and 3).



Example of non-isomorphic graphs where 3-WL test failed to distinguish (strongly regular graphs known as 4×4-Rook and Shrikhande graphs)

[1] Murphy, Srinivasan, Rao, Ribeiro, Relational pooling for graph representations, 2019

[2] Abboud, Ceylan, Grohe, Lukasiewicz, The surprising power of graph neural networks with random node initialization, 2020

[3] Balcilar, Heroux, Gauzere, Vasseur, Adam, Honeine, Breaking the limits of message passing graph neural networks, 2021

Experimental high expressivity

- We show empirically that graphs that cannot be distinguished by 1-WL^[1], 2-WL^[2] and 3-WL^[3] tests are easily distinguished by the proposed Graph ViT/MLP-Mixer (where all standard MP-GNNs fail).

Model	CSL (ACC)	EXP (ACC)	SR25 (ACC)
GCN	10.00 $\pm$ 0.00	51.90 $\pm$ 1.96	6.67 $\pm$ 0.00
GatedGCN	10.00 $\pm$ 0.00	51.73 $\pm$ 1.65	6.67 $\pm$ 0.00
GINE	10.00 $\pm$ 0.00	50.69 $\pm$ 1.39	6.67 $\pm$ 0.00
GraphTrans	10.00 $\pm$ 0.00	52.35 $\pm$ 2.32	6.67 $\pm$ 0.00
GCN-MLP-Mixer	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00
GatedGCN-MLP-Mixer	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00
GINE-MLP-Mixer	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00
GraphTrans-MLP-Mixer	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00

Table: Results are averaged over 4 runs with 4 different seeds.

[1] Murphy, Srinivasan, Rao, Ribeiro, Relational pooling for graph representations, 2019

[2] Abboud, Ceylan, Grohe, Lukasiewicz, The surprising power of graph neural networks with random node initialization, 2020

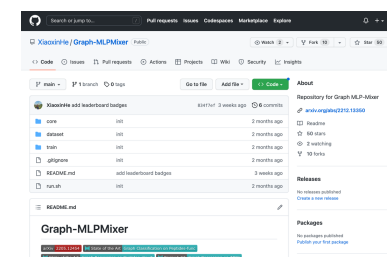
[3] Balcilar, Heroux, Gauzere, Vasseur, Adam, Honeine, Breaking the limits of message passing graph neural networks, 2021

Graph ViT/MLP-Mixer

- Graph ViT/MLP-Mixer
 - Offer an architecture at the boundary between standard MP-GNNs and Graph Transformers.
 - Improve MP-GNN limitations (poor long-range dependency and low expressivity power) and have better complexity than hybrid Transformers + Graph Transformers (linear complexity).
 - Demonstrate their potential on small and large molecular datasets.
- Reproducibility
 - GitHub : <https://github.com/XiaoxinHe/Graph-ViT-MLPMixer>
 - ArXiv : <https://arxiv.org/pdf/2212.13350.pdf>
 - ICML : <https://openreview.net/forum?id=l7yTbEWuOQ>

A GENERALIZATION OF ViT/MLP-MIXER TO GRAPHS

Xiaoxin He¹ Bryan Hooi¹ Thomas Laurent² Adam Perold³ Yann LeCun^{4,5} Xavier Bresson¹
{xiaoxin, bhooi, xavierca}@comp.nus.edu.sg, tlaurent@lms.edu
ap@discoverscience.com, yann@cs.nyu.edu
¹National University of Singapore ²Loyola Marymount University ³Element, Inc.
⁴New York University ⁵Meta AI



Outline

- Review of graph network architectures
- Graph positional encoding
- Transformers for sequences
- Graph Transformers
- ViT/MLP-Mixer for images
- Graph ViT/MLP-Mixer
- **Conclusion**

Conclusion

- Developments of GNNs
 - Vanilla GNNs $\Rightarrow$ Spectral GNNs $\Rightarrow$ Anisotropic GNNs $\Rightarrow$ WL-GNNs $\Rightarrow$ Graph Transformers $\Rightarrow$ Graph ViT
Best today
- What are the next potential architecture improvements?
 - Large-scale GNNs (with hierarchical property)
 - Computationally efficient GNNs, i.e. as fast as CNNs/Transformers (with standard hardware for dense linear algebra or new hardware for fast sparse linear algebra)



Questions?